L00 Start C programming in 15 minutes!

Hey, what's going on everybody?	嘿，大家好。
In this video, I'm going to teach you everything you need to know to start coding in C.	在这个视频中，我将教你开始用 C 语言编程所需的一切知识。
This video contains 12 hands-on projects, and for our final project, we will code a working clock.	这个视频包含 12 个动手项目，而我们的最终项目将编写一个可工作的时钟。
If that sounds good to you, I encourage you to sit back, relax, and enjoy the show.	如果你觉得不错，我鼓励你坐下来放松一下，好好欣赏。
I don't like boring introductions, so we're just going to jump right in.	我不喜欢无聊的介绍，所以我们直接开始。
To start coding in C, you'll need two things.	要开始用 C 语言编程，你需要两样东西。
An IDE, that's an integrated development environment.	一个 IDE，也就是集成开发环境。
It's a workspace in which we can write code.	它是一个我们可以编写代码的工作空间。
The other thing we'll need is a C compiler.	另一件我们需要的东西是一个 C 编译器。
It converts our written code into machine code so a computer can understand it.	它将我们编写的代码转换成机器码，以便计算机能够理解。
In this next section, we're going to install an IDE.	在下一节中，我们将安装一个 IDE。
In this next section, we're going to download Visual Studio Code.	在这一节中，我们将下载 Visual Studio Code。
It's a place in which we can write code.	它是一个我们可以编写代码的地方。
You can head to this URL code.visualstudio.com.	你可以访问这个网址：code.visualstudio.com。
Once you're here, go to this blue download button in the corner and then select the correct download for your operating system.	到达页面后，点击角落里的蓝色下载按钮，然后为你的操作系统选择正确的下载版本。
I'm running Windows, so I will download the Windows version.	我用的是 Windows，所以我会下载 Windows 版本。
And our download should start automatically.	然后下载应该会自动开始。
Now that our download is complete, we're going to open this executable.	现在下载完成，我们将打开这个可执行文件。
Read the license agreement.	阅读许可协议。
Yes, I did read it that fast.	是的，我确实读得那么快。
I accept the agreement.	我接受协议。
Next, you can select a destination folder.	下一步，你可以选择目标文件夹。
I'll keep it the same.	我会保持默认不变。
Next.	下一步。
Next.	下一步。
You can create a desktop icon if you would like.	你可以根据需要创建一个桌面图标。
I think I'll do so.	我想我会创建。
Next.	下一步。
And install.	然后安装。
And then just give it a moment.	然后稍等片刻。
We might as well launch Visual Studio Code right away.	我们不妨立即启动 Visual Studio Code。
I'll keep that checked.	我会保持勾选状态。
And finish.	然后完成。
Here we are within Visual Studio Code.	我们现在来到了 Visual Studio Code 里面。
We're going to create a new project folder.	我们将创建一个新的项目文件夹。
We can close out of this menu.	我们可以关闭这个菜单。
Go to the left toolbar.	转到左侧工具栏。
Go to the explore tab.	转到资源管理器选项卡。
And we will open a folder and or create a new one.	我们将打开一个文件夹，或者创建一个新文件

夹。
Pick a place in which you would like to create a new folder.
选择一个你想要创建新文件夹的位置。
I'll do so on my desktop.
我会在桌面上创建。
I'm going to right-click, go to new folder.
我将右键单击，选择新建文件夹。
I'll call this folder "coding", but feel free to pick something else if you would like.
我会把这个文件夹命名为 coding，但如果你愿意，也可以选别的名字。
Click on your folder, then click this button of select folder.
点击你的文件夹，然后点击“选择文件夹”按钮。
Now, we're going to create a new C file.
现在，我们将创建一个新的 C 文件。
Within your folder, we're going to go to new file.
在你的文件夹中，我们点击新建文件。
A common naming convention for a C file is "main" because it holds the main body of your code.
C 文件的一个常见命名约定是 main，因为它包含代码的主体。
main.c. Enter.
main.c，然后回车。
We're within an empty C file.
我们进入了一个空的 C 文件。
There's a few extensions I would recommend.
有几个扩展我推荐安装。
Let's go to the left toolbar.
我们转到左侧工具栏。
Go to extensions.
转到扩展。
And we are going to look up the C/C++ extension.
然后我们搜索 C/C++ 扩展。
C/C++ extension pack.
C/C++ 扩展包。
This will give us some quality of life improvements for C programming.
这将为我们提供一些 C 语言编程的便利改进。
So, be sure to download this extension.
所以请务必下载这个扩展。
Once this extension is downloaded, the next extension is the Code Runner extension.
这个扩展下载完成后，下一个扩展是 Code Runner。
So, look up Code Runner and install.
所以搜索 Code Runner 并安装。
Okay, we can close out of this extensions menu and go back to the Explorer window.
好的，我们可以关闭这个扩展菜单，回到资源管理器窗口。
For the Code Runner extension, there's two settings I recommend enabling before we continue.
对于 Code Runner 扩展，有两个设置我建议在继续之前启用。
Let's go to file, preferences, settings.
我们前往 文件 > 首选项 > 设置。
We're going to look up code runner.
我们将搜索 code runner。
Check this check box of "clear previous output".
勾选“clear previous output”这个复选框。
That'll clear the output before we run each program.
这将在每次运行程序之前清除之前的输出。
And then scroll down, check "save file before run".
然后向下滚动，勾选“save file before run”。
So you don't have to manually save each time we want to run a program.
这样你就不用在每次运行程序时手动保存了。
And those are the two settings I would recommend for the Code Runner extension.
这就是我为 Code Runner 扩展推荐的两个设置。
Next on our to-do list is to check to see if we have a C compiler already installed.
我们待办事项列表的下一项是检查我们是否已经安装了 C 编译器。
To do that in VS Code, you can go to the terminal tab.
在 VS Code 中，你可以转到终端选项卡。
Go to new terminal.
点击新建终端。
Now, the instructions are going to vary based on your operating system.
现在，具体操作步骤会根据你的操作系统有所不同。
Currently, I'm running Windows.
当前我用的是 Windows。
To check to see if you have a C compiler using Windows, you'll type "gcc --version".
要在 Windows 上检查是否安装了 C 编译器，你需要输入 gcc --version。
If you don't have one installed and set up, you'll see an error message like this.
如果你没有安装和配置好，你会看到类似这样的错误信息。
The term 'gcc' is not recognized.
“术语 'gcc' 未被识别。”

But if you're on Windows and you do have one installed, you'll see something like this.
gcc --version, and here's my compiler, gcc.exe, and then I have a version number and some other copyright stuff.
Let's clear this terminal.
If you need to set up the C compiler for Windows, it's actually really complicated.
Look in the description of this video for any timestamps and you can skip ahead.
If you're a Mac user, you're going to type "clang --version".

This is how to check to see if you have a C compiler using a Mac.
But since I'm on Windows, this term isn't recognized.
To install the C compiler on Mac, it's actually pretty simple: type "xcode-select --install".
This will install the C compiler and some dev tools.
But again, I'm on Windows; these terms aren't recognized.

If you're a Linux user, to check to see if you have a C compiler, you'll type "gcc --version".
If you're on Linux and you don't have a C compiler set up, you'll type "sudo apt-get update".
This will refresh the package list.
Hit enter.
Then to install the C compiler, you'll type (if you're on Linux) "sudo apt-get install build-essential gdb".
This will install the compiler, make, and debugger.
Again, this is if you're a Linux user.
So, if you're on Windows, we need to set up the Windows installation.
Our next step is that we have to download a compiler.
It will translate our written code into machine code.
To make this as easy as possible, I recommend going to this website, msys2.org.
MSYS2 is a collection of tools and libraries for building, installing, and running native Windows software.
We can really just follow the installation instructions.
The compiler is going to be provided with this.
We'll download the installer.
Then we just have to wait for it.
Also, please note to run the installer, this requires 64-bit Windows 10 or newer.
Once the download is complete, we'll open this executable.
Click next.
Select a destination folder.
I'll keep it as is within my C drive.
Next.
You can create a start menu shortcut.

但如果你在 Windows 上已经安装了编译器，你会看到类似这样的信息。
gcc --version，这是我的编译器 gcc.exe，然后有版本号和一些版权信息。
我们清空这个终端。
如果你需要为 Windows 设置 C 编译器，实际上非常复杂。
请查看本视频描述中的时间戳，你可以跳过相关部分。
如果你是 Mac 用户，你需要输入 clang --version。
这就是在 Mac 上检查是否安装了 C 编译器的方法。
但由于我用的是 Windows，这个命令未被识别。
在 Mac 上安装 C 编译器实际上非常简单：输入 xcode-select --install。
这将安装 C 编译器和一些开发工具。
但再说一次，我用的是 Windows；这些命令未被识别。
如果你是 Linux 用户，要检查是否安装了 C 编译器，你需要输入 gcc --version。
如果你在 Linux 上并且没有安装 C 编译器，你需要输入 sudo apt-get update。
这将刷新软件包列表。
按回车。
然后要安装 C 编译器，你需要输入（如果你在 Linux 上）sudo apt-get install build-essential gdb。
这将安装编译器、make 和调试器。
再次说明，这是针对 Linux 用户的。
所以，如果你用的是 Windows，我们需要进行 Windows 安装设置。
我们的下一步是必须下载一个编译器。
它会将我们编写的代码翻译成机器码。
为了尽可能简化，我推荐访问这个网站：msys2.org。
MSYS2 是一个用于构建、安装和运行原生 Windows 软件的工具和库的集合。
我们基本上只需要按照安装说明操作即可。
编译器会随之提供。
我们将下载安装程序。
然后我们只需等待。
另外请注意，运行此安装程序需要 64 位的 Windows 10 或更新版本。
下载完成后，我们会打开这个可执行文件。
点击下一步。
选择一个目标文件夹。
我会保持原样，放在我的 C 盘里。
下一步。
你可以创建开始菜单快捷方式。

I'll just press next.	我就按下一步。
And we are installing.	然后我们开始安装。
We'll run MSYS2 now.	我们现在运行 MSYS2。
Keep this box checked and finish.	保持勾选这个框，然后点击完成。
We'll use this piece of software MSYS2 to download the C compiler.	我们将使用 MSYS2 这个软件来下载 C 编译器。
Here's how.	方法如下。
Now according to our instructions, the next step is that we have to type this command.	现在根据我们的说明，下一步是输入这个命令。
Honestly, we can just copy this.	老实说，我们可以直接复制。
Copy and paste.	复制并粘贴。
pacman means package manager, not the arcade game.	pacman 是包管理器的意思，不是那个街机游戏。
Dash capital S for sync.	短横线加大写 S 表示同步（Synchronize）
And here's the name of the package.	这是软件包的名称。
It's really long.	它真的很长。
Then I'll hit enter to submit it.	然后我按回车提交。
Proceed with the installation.	继续安装。
Capital Y for yes.	大写 Y 表示是。
And this will take a moment.	这会花一点时间。
We're downloading all the packages we'll need.	我们正在下载所有需要的软件包。
Let's ensure that our compiler downloaded successfully.	我们来确认一下编译器是否下载成功。
You can type "gcc --version".	你可以输入 gcc --version。
It should display the current version of the C compiler that we're using.	它应该会显示我们正在使用的 C 编译器的当前版本。
All right, we can close out of this window.	好了，我们可以关闭这个窗口。
And now we need to set the variable path.	现在我们需要设置路径变量。
Let's locate that folder.	我们来找到那个文件夹。
It's within my C drive, MSYS64.	它在我的 C 盘里，MSYS64。
We need to find the C compiler.	我们需要找到 C 编译器。
It should be underneath this folder of UCRT64.	它应该在这个 UCRT64 文件夹下面。
Go to bin and we are looking for this file.	进入 bin 目录，我们正在寻找这个文件。
We're going to copy the file location.	我们将复制文件位置。
I'll right-click, go to properties, and copy this location.	我会右键单击，进入属性，然后复制这个位置。
What we got to do next is set the PATH variable.	我们接下来要做的是设置 PATH 变量。
We have to let our operating system know where it can find the C compiler, that GCC file.	我们必须让操作系统知道它在哪里可以找到 C 编译器，也就是那个 GCC 文件。
We're going to do a search for "environment variables".	我们将搜索“环境变量”。
Edit the system environment variables.	编辑系统环境变量。
Click environment variables.	点击环境变量。
Select Path.	选择 Path。
Edit.	编辑。
New.	新建。
Then paste that path, and it should be to that GCC file.	然后粘贴那个路径，它应该指向那个 GCC 文件。
Press okay.	按确定。
Then okay.	然后确定。
Okay.	好的。
And that's it.	就这样。
We have downloaded and set up the C compiler from Windows.	我们已经从 Windows 上下载并设置好了 C 编译器。

All right.
好的。
We're ready to write our first C program within our C file.
我们准备好要在我们的 C 文件中编写第一个 C 程序了。

The first thing we're going to write is this statement: #include <stdio.h>.
我们要写的第一条语句是：#include <stdio.h>。
This statement is a pre-processor directive.
这条语句是一个预处理指令。
We're telling the compiler to include the standard input output library.
我们告诉编译器要包含标准输入输出库。
It contains a lot of useful functions for output and input.
它包含很多用于输入输出的有用函数。
We need this header file in order to display text to the screen.
我们需要这个头文件才能将文本显示到屏幕上。
Later on in this program, we need a main function in order for this program to well function.
在这个程序中，我们还需要一个 main 函数，这样程序才能正常运行。
Here's how to create the main function.
下面是创建 main 函数的方法。
You'll type int, meaning integer.
你会输入 int，表示整数。
The entry point for a program is the main function.
程序的入口点是 main 函数。
Main. Add a set of parentheses, then a set of curly braces.
Main，加一对圆括号，再加一对花括号。
Your program is not going to run without the main function.
没有 main 函数，你的程序将无法运行。
We can even try it.
我们甚至可以试一下。
I'm going to cut it temporarily.
我会暂时把它剪切掉。
To run your C program, go to this arrow and select run code.
要运行你的 C 程序，点击这个箭头并选择 run code。

There's also a shortcut if you would like to use that, too.
如果你想使用快捷键，也有一个快捷键。
Well, without the main function, we get this error message: "returned 1 exit status".
如果没有 main 函数，我们会得到这条错误信息："returned 1 exit status"。
I'll paste the main function back in.
我会把 main 函数粘贴回去。
Now, we don't get that error message.
现在，我们不会得到那个错误信息了。
And we do have an executable within our coding folder.
而且我们的 coding 文件夹里确实有了一个可执行文件。

Our program is running successfully, but it doesn't do anything.
我们的程序运行成功了，但它什么也没做。
So, you need the main function in order for your program to well function.
所以，你需要 main 函数才能让程序正常运行。
It's the entry point of your program.
它是程序的入口点。
What you'll commonly see at the end of the main function is this statement: return 0;
在 main 函数的末尾你通常会看到这条语句：return 0;
In the C programming language, the main function is expected to return an integer.
在 C 语言中，main 函数需要返回一个整数。
It returns it to the operating system.
它将其返回给操作系统。
This integer serves as an exit code.
这个整数充当退出代码。
Zero indicates that the program ran successfully.
0 表示程序成功运行。
A nonzero value typically indicates an error or problem of some kind.
非零值通常表示某种错误或问题。

In older versions of C such as C89 or C90, omitting this statement will lead to undefined behavior.
在 C 语言的老版本（如 C89 或 C90）中，省略这条语句会导致未定义行为。
But in later standards, you can omit the statement.
但在后来的标准中，你可以省略这条语句。
But it's considered good practice to include this return statement just for backwards compatibility.
但为了向后兼容，包含这条 return 语句被认为是一种良好实践。
We have a program, but it doesn't do anything.
我们有一个程序，但它什么也不做。
So let's print some text.
所以让我们打印一些文本。
The main function is the entry point of our program.
main 函数是我们程序的入口点。

Within the set of curly braces, we'll write some code that we want to print.
在一对花括号内，我们将编写想要打印的代码。

To print some code, you can use the printf statement.
要打印一些内容，你可以使用 printf 语句。

printf("");, then a semicolon at the end.
printf("");，然后在末尾加分号。

We end all of our statements with a semicolon.
我们所有的语句都以分号结尾。

It's kind of like the period at the end of a sentence in the English language.
这有点像英语中句子末尾的句号。

Whatever you would like to print to the screen, you're going to do so within quotes, double quotes.
任何你想打印到屏幕上的内容，都要放在双引号内。

Think of a food you like.
想一个你喜欢的食物。

My first statement will be "I like pizza!"
我的第一条语句将是 "I like pizza!"

And now if we were to run this program, we can print this text.
现在如果我们运行这个程序，我们就可以打印这段文字。

So let's run this code.
所以让我们运行这段代码。

And the output is "I like pizza."
输出结果是 "I like pizza."

Let's add a second printf statement.
让我们添加第二个 printf 语句。

printf("It's really good."); then be sure to include that semicolon at the end.
printf("It's really good."); 然后确保末尾加上分号。

Now we should have two lines of output.
现在我们应该有两行输出。

I like pizza. It's really good. — one issue that we're running into is that we're displaying all of the text in one line.
I like pizza. It's really good. — 我们遇到的一个问题是所有文本都显示在同一行。

To move down to the next line, you can add a newline character, which is a forward slash and an N.
要换到下一行，你可以添加一个换行符，即一个反斜杠和一个 n。backslash

That's a newline character.
那就是换行符。

Now, our next line of output should be on the next line.
现在，我们的下一行输出应该在新的一行上。

I like pizza. It's really good.
I like pizza. It's really good.

To print some text to the screen, you just type printf, then within quotes, add some text.
要向屏幕打印文本，你只需输入 printf，然后在引号内添加一些文本。

Now we're going to discuss comments.
现在我们讨论注释。

To write a comment in C, you use two forward slashes.
在 C 语言中写注释，你使用两个正斜杠。

Then you can write some text.
然后你可以写一些文字。

For example, let's say "This is my first program."
例如，假设写 "This is my first program."

Comments won't display as output.
注释不会显示为输出。

And you can see that the text color is green, but that's a feature of VS Code.
你可以看到文本颜色是绿色的，但那是 VS Code 的功能。

If I were to run this program, we don't see this comment.
如果我运行这个程序，我们不会看到这个注释。

We still have our output: "I like pizza. It's really good."
我们仍然有输出："I like pizza. It's really good."

Comments are useful as notes for yourself or for other developers.
注释作为给自己或其他开发者的笔记非常有用。

You can also write a multi-line comment.
你也可以写多行注释。

To start a multi-line comment, use a forward slash, then an asterisk.
开始一个多行注释，使用一个正斜杠，然后一个星号。

You'll have an asterisk, then a forward slash again, wherever the comment ends.
在注释结束的地方，再放一个星号，然后一个正斜杠。

We could say something like "This is also a comment."
我们可以说 "This is also a comment."

This won't display as output either.
这也不会显示为输出。

We still have "I like pizza. It's really good."
我们仍然有 "I like pizza. It's really good."

Throughout the series, I'll be writing a lot of comments.
在整个系列中，我会写很多注释。

And that's your first lesson in C.
这就是你关于 C 语言的第一课。

Now, you have a homework assignment.
In the comment section down below, you have to write two comments and two print statements.
If you don't do your homework, I'm going to call the police and they're going to come to your house and arrest you.
And well, everybody, that is your first lesson in C programming.

现在，你有一个家庭作业。
在下面的评论区，你必须写两个注释和两个 printf 语句。
如果你不做作业，我会打电话给警察，他们会来你家逮捕你。
嗯，各位，这就是你们关于 C 语言编程的第一课。

```c
#include <stdio.h>
int main(){
    // DO YOUR HOMEWORK
    /*
    I
    WILL
    KNOW
    IF
    YOU
    DON'T
    */
    printf("I like pizza!\n");
    printf("It's really good!");
    return 0;
}
```

本章红色单词原型：integrate, workspace, compiler, convert, automatically, executable, license, icon, launch, toolbar, tab, folder, convention, extension, preference, previous, scroll, manually, terminal, instruction, vary

L02 Learn C programming variables easy!

All right everybody, in this video I'm going to explain variables in the C programming language.
You're probably used to variables back in middle school; a variable represents some value.
Well in programming we have something similar: a variable is a reusable container for a value.
The variable behaves as if it were the value it contains, but we do have the capability of storing more than just numbers; there's different data types.
Let's begin with the data type of int, meaning integer.
To create an integer variable, we will say int (meaning integer), then we need a unique name for this variable.
Let's say we're working with ages; let's say int age; we'll assign this variable a value using equals.
So let's say that I'm 25.
In the C programming language, we have to end each statement with a semicolon.
So we can use this variable age, and it will behave as if it was the number 25.
To display this variable, we can use that within a print statement.
So printf in the C programming language.
To display a variable, we'll have to use a format specifier.
Let's say we have a string of text such as "You are ? years old".

好的，各位，在这个视频中，我将讲解 C 语言中的变量。
你们可能在中学就接触过变量；变量代表某个值。
在编程中，我们有类似的概念：变量是一个可重复使用的值容器。
变量的行为就好像它包含的值本身，但我们不仅可以存储数字；还有不同的数据类型。
我们从 int 数据类型开始，它表示整数。
要创建一个整数变量，我们写 int（表示整数），然后需要给这个变量起一个唯一的名字。
假设我们处理年龄；写 int age；我们用等号给这个变量赋值。
假设我 25 岁。
在 C 语言中，每条语句必须以分号结尾。
这样我们就可以使用变量 age，它的行为就好像它是数字 25 一样。
要显示这个变量，我们可以在 printf 语句中使用它。
所以用 C 语言中的 printf。
要显示变量，我们必须使用格式说明符。
假设我们有一段文本，比如 "You are ? years

old"。

To insert a variable, I'm going to use a percent sign for a format specifier; to display an int (an integer), we'll use "d" meaning decimal.

要插入一个变量，我使用百分号作为格式说明符；要显示 int（整数），我们使用 "d"，表示十进制。

years old. Following the string of text, I will add a comma, then the variable I would like to insert: my variable age.

years old。在文本字符串后面，我加一个逗号，然后是要插入的变量：我的变量 age。

Let's give it a test run: "You are my variable age which is 25 years old."

我们来测试运行一下："You are my variable age which is 25 years old."

So an int is a whole number, a whole integer.

所以 int 是整数，是一个完整的整数。

If I were to say 25.5 (you know, like it's my half birthday), well we end up truncating that decimal portion because integers can't store decimals.

如果我说 25.5（比如我的半岁生日），那么小数部分会被截断，因为整数不能存储小数。

To store a whole number, you can use an integer.

要存储整数，你可以使用 int。

Let's create another variable; repetition is going to help us remember these steps.

我们再创建另一个变量；重复有助于我们记住这些步骤。

Another example of an integer could be a year.

另一个整数的例子可以是年份。

int year; what year is it? I will say 2025.

int year; 今年是哪一年？我写 2025。

Then I'll create another printf statement: printf("The year is %d\n", year);

然后我再写一个 printf 语句：printf("The year is %d\n", year);

The year is 2025.

年份是 2025。

Let's create another. What's another good example of an integer? Perhaps a quantity.

我们再创建一个。整数的另一个好例子是什么？也许是数量。

Maybe I'm buying just one product; you wouldn't have half a product, right? Quantities would be whole numbers typically speaking.

假设我只买一件商品；你不会买半件商品，对吧？数量通常都是整数。

So let's say printf("You have ordered %d items\n", quantity);

所以我们写 printf("You have ordered %d items\n", quantity);

You have ordered 1 item(s).

你订购了 1 件商品。

And then we can change these values too; maybe I would like to buy two items for the quantity.

而且我们还可以修改这些值；也许我想把数量改成 2 件。

You have ordered 2 items.

你订购了 2 件商品。

Alright, so those are integers; they're just whole numbers; they don't have the capability of storing decimal portions.

好了，这些就是整数；它们只是整数；不能存储小数部分。

That's where the next data type is going to come in.

这就需要下一个数据类型了。

To store a decimal portion, you can use a float (meaning floating point number).

要存储小数部分，你可以使用 float（浮点数）。

What's a good example of a number that contains a decimal? Perhaps a GPA, a grade point average.

包含小数的数字有什么好例子？也许是 GPA，平均绩点。

float GPA; I will set this to 2.5.

float GPA; 我把它设为 2.5。

Hey, C's get degrees, right?

嘿，得 C 也能拿学位，对吧？

Then to output this, I would use printf.

然后要输出它，我会使用 printf。

Let's think of a sentence: "Your GPA is ".

我们想一个句子："Your GPA is "。

Use a format specifier; then for a floating point number, we're going to type "f" for floating point number.

使用格式说明符；对于浮点数，我们用 "f"。

I'll add a newline character because I'm probably going to forget to do that later.

我会加上换行符，因为我之后很可能会忘记。

And then I will display my variable of GPA.

然后我会显示我的变量 GPA。

Your GPA is 2.500000.

Your GPA is 2.500000。

C has a default behavior of displaying six digits after the decimal

C 语言在处理浮点数时默认会显示小数点后六

when working with floating point numbers.
位。
We do have the capability of changing that, but that's going to be a future video.
我们确实有办法改变它，但那将是以后视频的内容。
You could add a dot and then a number to display a given amount of digits after the decimal.
你可以加点再加一个数字来指定小数点后显示的位数。
Your GPA is 2.5 — but that's a whole another topic; we'll cover that in a future video.
Your GPA is 2.5 —— 但那完全是另一个话题；我们将在以后的视频中讲解。
Another good example of a float could be a price.
另一个 float 的好例子是价格。
float price = 19.99; like $19.99.
float price = 19.99; 比如 19.99 美元。
We will use printf("The price is $%.2f\n", price); — I'll pick a unit of currency, I'll pick American dollars, but pick whatever you would like.
我们使用 printf("The price is $%.2f\n", price); —— 我选一个货币单位，我选美元，但你可以随意选。
To insert a variable, we'll use that format specifier: percent sign then "f" for a floating point number.
要插入变量，我们使用格式说明符：百分号然后 "f" 表示浮点数。
Then I'll add a newline character because I'm probably going to forget if I don't.
然后我加上换行符，因为如果不加我可能会忘记。
The price is $19.99.
The price is $19.99。
To display two digits after the decimal, you can use ".2f". More on that in a future video.
要显示小数点后两位，你可以使用 ".2f"。更多内容在以后的视频中。
A temperature could be a floating point number.
温度也可以是一个浮点数。
float temperature = -10.1; it's really cold outside.
float temperature = -10.1; 外面真的很冷。
We'll use printf("The temperature is at %.1f\n", temperature);
我们使用 printf("The temperature is at %.1f\n", temperature);
Then I'll add a newline character, then display our temperature variable.
然后我加上换行符，再显示我们的温度变量。
The temperature is -10.1.
The temperature is -10.1。
This can be Celsius or Fahrenheit. Let's add that in.
这可以是摄氏度或华氏度。我们加上。
To add a degree symbol on Windows, hold Alt then type 0176. I'll say "F" for Fahrenheit.
要在 Windows 上添加度符号，按住 Alt 然后输入 0176。我说 "F" 表示华氏度。
The temperature is -10.1 degrees Fahrenheit.
The temperature is -10.1 degrees Fahrenheit。
Then you can set the precision if you would like, but we're not at that point yet.
然后你可以设置精度，但我们还没到那一步。
Alright, so those are floating point numbers; they're numbers including a decimal portion.
好了，这些就是浮点数；它们是包含小数部分的数字。
We only have the capability of storing six to seven digits after the decimal.
我们只能存储小数点后六到七位数字。
If you need more precision, such as if you're working with pi, you can use a different data type known as a double.
如果你需要更高的精度，比如处理圆周率 π，你可以使用另一种叫做 double 的数据类型。
So let's say double pi = 3.14159; then I don't know the rest, so I have to look them up.
我们写 double pi = 3.14159; 后面的我不记得了，所以我得查一下。
So with doubles, you can store 15 to 16 digits after the decimal.
使用 double，你可以存储小数点后 15 到 16 位数字。
And then to display a double, we'll use printf("The value of pi is %.15lf\n", pi);
然后要显示 double，我们使用 printf("The value of pi is %.15lf\n", pi);
We'll use a format specifier, then type "lf" meaning long float.
我们使用格式说明符，然后写 "lf" 表示 long float。
Then we will display our variable pi. Now again, the default behavior of C is to display six digits after the decimal; you'd have to use a dot followed by the amount of digits you would want to
然后显示变量 pi。再次说明，C 的默认行为是显示小数点后六位；你需要用点加数字来指定要显示的位数。

display.
Another example could be Euler's number; this I got to look up. I'll just copy this.
另一个例子可以是欧拉数；这个我也得查一下。我就复制一下。
The value of e is %.15lf\n, e; then add a newline character.
The value of e is %.15lf\n, e; 然后加上换行符。
The value of pi is 3.141590000000000 — 3.14 and some change; "the value of e is 2.718281828459045" — 2.7 and some change.
π 的值是 3.141590000000000 —— 3.14 多一点；"e 的值是 2.718281828459045" —— 2.7 多一点。
If you have a decimal number that needs high precision, you can use a double.
如果你有一个需要高精度的小数，可以使用 double。
In this series, we'll probably not be working with doubles too often; we'll stick with floats.
在这个系列中，我们可能不会太常用 double；我们会主要使用 float。
But at this level, you should still be aware of the existence of doubles.
但在这个阶段，你还是应该知道 double 的存在。
With variables, you can store more than just numbers; you can store characters.
使用变量，你不仅可以存储数字，还可以存储字符。
Another data type is char, meaning character.
另一种数据类型是 char，表示字符。
Let's say we have a letter grade, like what grade did you get on the exam?
假设我们有一个字母等级，比如你考试得了什么等级？
When working with characters, you need to use a set of single quotes.
处理字符时，你需要使用一组单引号。
Let's say we got an 'A' on the exam; that's our grade.
假设我们考试得了 A；那就是我们的成绩。
Then to display this, let's use printf("Your grade is %c\n", grade);
然后显示它，我们使用 printf("Your grade is %c\n", grade);
Your grade is A. Or if I failed, this could be an 'F'. "Your grade is F".
Your grade is A。或者如果我挂了，可以是 F。"Your grade is F"。
Let's think of another example; perhaps a symbol.
再想一个例子；也许是一个符号。
char symbol = '!'; I'll pick an exclamation point; that's the first thing that came to mind.
char symbol = '!'; 我选感叹号；这是首先想到的。
Let's print our symbol. What can we say? I can't really think of anything, so let's say "Your favorite symbol is %c\n", symbol.
我们来打印这个符号。说什么好呢？我实在想不出，那就说 "Your favorite symbol is %c\n", symbol。
Your favorite symbol is an exclamation point.
Your favorite symbol is an exclamation point.
What about currency? char currency = '$'; I'll pick American dollars.
货币呢？char currency = '$'; 我选美元。
printf("The currency is %c\n", currency); "The currency is $".
printf("The currency is %c\n", currency); "The currency is $"。
Alright, those are chars; they store a single character.
好了，这些就是 char；它们存储单个字符。
If you've programmed in a different language before, you're probably familiar with strings.
如果你之前用过其他编程语言，你可能对字符串很熟悉。
Well in C, we don't have strings like in other languages. A string is a series of characters, such as a name.
在 C 语言中，我们没有其他语言那种字符串。字符串是一系列字符，比如名字。
In the C programming language, we use an array of characters to represent a string.
在 C 语言中，我们使用字符数组来表示字符串。
So to store more than one character, we can use char again (because we're working with characters).
所以要存储多个字符，我们再次使用 char（因为我们处理的是字符）。
Think of a variable name such as name, then add a set of square brackets; this is an array.
想一个变量名比如 name，然后加一对方括号；这就是数组。
An array can store more than one value. So to set it, we'll use a set of double quotes.
数组可以存储多个值。所以要赋值，我们使用一组双引号。
So now we can store more than one character. Why don't you type
现在我们可以存储多个字符了。你为什么不输

in your name? I'll use my YouTube channel name.
入你的名字呢？我用我的 YouTube 频道名。

Alright, then to print our string (well our array of characters, technically), we'll say printf("Hello %s\n", name);
好了，然后要打印我们的字符串（技术上说是字符数组），我们写 printf("Hello %s\n", name);

Hello whatever your name is, whatever you typed here.
Hello 加上你的名字，你在这里输入的内容。

With our array of characters (also known as a string in other languages), you can store more than one character, and again they do have to be within double quotes.
使用字符数组（在其他语言中也称为字符串），你可以存储多个字符，并且它们必须放在双引号内。

So what's another example of an array of characters? Perhaps a food, the name of a food.
那么字符数组的另一个例子是什么？也许是食物，食物的名称。

char food[] = "pizza";
printf("Your favorite food is %s\n", food);

Hello bro code, your favorite food is pizza.
Hello bro code, your favorite food is pizza.

What about an email? char email[] = "fake123@gmail.com";
电子邮件呢？char email[] = "fake123@gmail.com";

Now with strings, you can include numbers, but they're treated as characters rather than numeric values, so we can't use them with any sort of math.
在字符串中，你可以包含数字，但它们会被当作字符而不是数值，所以不能用于任何数学运算。

In the future, we have a character representation of the numbers 1,2,3.
以后，我们会用字符来表示数字 1,2,3。

Then I'll add an extension of "@gmail.com". I don't want to give you guys my real email because some of you guys are weird, no offense.
然后我加上 "@gmail.com"。我不想给你们我的真实邮箱，因为你们有些人有点奇怪，没有冒犯的意思。

Your email is %s\n, email;
Your email is %s\n, email;

Hello bro code, your favorite food is pizza; your email is fake123@gmail.com.
Hello bro code, your favorite food is pizza; your email is fake123@gmail.com.

One last data type we're going to cover is bool, meaning boolean.
我们要讲的最后一个数据类型是 bool，表示布尔值。

Booleans are either true or false.
布尔值要么是 true，要么是 false。

But to work with booleans, we have to include this header file at the top: #include <stdbool.h> (meaning standard boolean).
但要使用布尔值，我们必须在顶部包含这个头文件：#include <stdbool.h>（表示标准布尔）。

And this is a header file. To create a boolean variable, we'll type bool.
这是一个头文件。要创建布尔变量，我们写 bool。

Bools are binary; they're either true or false. True means 1, false means 0. There's only two possible values.
布尔是二进制的；要么 true 要么 false。True 表示 1，false 表示 0。只有两种可能的值。

This could be good if determining if somebody is online, for example.
比如，判断某人是否在线就很有用。

We'll create a variable of "is online": bool isOnline = true;
我们创建一个变量 isOnline：bool isOnline = true;

They're either online or they're offline. So let's say that this is true.
他们要么在线，要么离线。所以我们说这个是 true。

So true also corresponds to the number 1; that's also valid.
所以 true 也对应数字 1；那也是有效的。

Or if somebody was offline, this could be false; this also corresponds to the number 0.
或者如果有人离线，这个可以是 false；这也对应数字 0。

We tend to use booleans more internally within a program, such as when working with if statements, but you should still be familiar with them at this level.
我们通常更多地在程序内部使用布尔值，比如处理 if 语句时，但在这个阶段你还是应该熟悉它们。

If I was to display a boolean variable, this is what it would look like. So I'm just going to use a format specifier %d.
如果我想要显示一个布尔变量，看起来会是这样。所以我用格式说明符 %d。

We will display isOnline: printf("%d\n", isOnline); this would give me the number 1.
我们会显示 isOnline：printf("%d\n", isOnline); 这会给我数字 1。

1 also means true. If somebody were offline, let's set that to be
1 也表示 true。如果有人离线，我们设为 false；

false; this would be 0.
这会输出 0。

With booleans, we tend to not display them directly using printf; you would more likely see them be used internally within a program, such as within an if statement.
对于布尔值，我们通常不会直接用 printf 显示它们；你更可能看到它们在程序内部使用，比如在 if 语句中。

If statements are a future topic, but I just want to give you a quick demonstration.
if 语句是以后的话题，但我只想快速演示一下。

You would type if (isOnline) { printf("You are online\n"); } else { printf("You are offline\n"); }
你会写 if (isOnline) { printf("You are online\n"); } else { printf("You are offline\n"); }

Let's give it a test run. Our boolean of isOnline is true; since this is true, we would print "You are online".
我们来测试运行一下。我们的布尔值 isOnline 是 true；因为这是 true，我们会打印 "You are online"。

The number 1 can also represent true; "You are online".
数字 1 也可以表示 true；"You are online"。

Or if this were false, well then we would execute this code: "You are offline".
或者如果这个是 false，那么我们会执行这段代码："You are offline"。

False also corresponds with 0; "You are offline".
False 也对应 0；"You are offline"。

However, as humans, using true or false is more understandable to us. By looking at this code, we may not know what 1 means, but true or false makes more sense.
然而，作为人类，使用 true 或 false 对我们来说更容易理解。看这段代码时，我们可能不知道 1 是什么意思，但 true 或 false 更有意义。

Okay, let's do another example. Let's say bool isStudent = true;
好的，我们再做一个例子。假设 bool isStudent = true;

If a student, if that's true, "You are a student", else "You are not a student" (maybe you're alumni).
如果是学生，如果为 true，则 "You are a student"，否则 "You are not a student"（也许你是校友）。

Don't forget that semicolon. "You are a student". If this were false, then "You're not a student".
别忘了分号。"You are a student"。如果这个是 false，则 "You're not a student"。

One last example: bool forSale = false; If forSale, "That item is for sale", else "That item is not for sale".
最后一个例子：bool forSale = false; 如果 forSale，则 "That item is for sale"，否则 "That item is not for sale"。

That item is not for sale. If it were true, then "The item is for sale".
That item is not for sale。如果为 true，则 "The item is for sale"。

So booleans are either true or false. True also means 1, false also means 0, but that's less readable to a human.
所以布尔值要么是 true 要么是 false。True 也代表 1，false 也代表 0，但这对人类来说可读性较差。

So to work with booleans, just include this header file.
所以要使用布尔值，只需包含这个头文件。

Alright everybody, so in summary: variables are reusable containers for a value; a variable behaves as if it were the value it contains.
好了各位，总结一下：变量是可重用的值容器；变量的行为就好像它包含的值本身。

Integers store whole numbers; they tend to be 4 bytes (at least in modern systems).
整数存储整数值；它们通常是 4 字节（至少在 modern 系统中是这样）。

Floating point numbers are single-precision decimal numbers; they take up 4 bytes.
浮点数是单精度小数；它们占用 4 字节。

A double has double the precision of a float; they use 8 bytes.
double 的精度是 float 的两倍；它们使用 8 字节。

A char is a single character; they use up 1 byte.
char 是单个字符；占用 1 字节。

Now an array of characters (also known as a string): you can store words or whole sentences; the size varies because the strings can vary in length.
字符数组（也称为字符串）：你可以存储单词或整个句子；大小可变，因为字符串长度可以不同。

And then a bool (meaning boolean) stores either true or false; they use up 1 byte, but they require the standard boolean header
然后 bool（布尔值）存储 true 或 false；占用 1 字节，但需要标准布尔头文件。

file.
We'll have a lot more practice with variables.
Now your assignment is to post an integer, a float, a char, an array of characters, and a boolean in the comment section below.

That's your homework. And if you don't do your homework, I'm going to call the police.
And well, everybody, those are variables in C.

我们会有更多关于变量的练习。
现在你的作业是在下面的评论区贴出一个整数、一个 float、一个 char、一个字符数组和一个布尔值。
这就是你的作业。如果你不做作业，我就报警。

嗯，各位，这就是 C 语言中的变量。

```c
#include <stdio.h>
#include <stdbool.h>
int main() {
    // variable = A reusable container for a value.
    //                Behaves as if it were the value it contains.
    // int = whole numbers (4 bytes in modern systems)
    // float = single-precision decimal number (4 bytes)
    // double = double-precision decimal number (8 bytes)
    // char = single character (1 byte)
    // char[] = array of characters (size varies)
    // bool = true or false (1 byte, requires <stdbool.h>)
    int age = 21;
    int year = 2025;
    int quantity = 1;
    printf("You are %d years old\n", age);
    printf("The year is %d\n", year);
    printf("You have ordered %d x items\n", quantity);
    float gpa = 2.5;
    float price = 19.99;
    float temperature = -10.1;
    printf("The price is $%f\n", price);
    printf("Your gpa is %f\n", gpa);
    printf("The temperature is %f°F\n", temperature);
    double pi = 3.141592653589793;
    double e = 2.7182818284590;
    printf("The value of pi is %lf\n", pi);
    printf("The value of e is %lf\n", e);
    char grade = 'A';
    char symbol = '!';
    char currency = '$';
    printf("Your letter grade is %c\n", grade);
    printf("%c is your favorite symbol\n", symbol);
    printf("The currency is in %c\n", currency);

    char name[] = "Bro Code";
    char food[] = "pizza";
    char email[] = "fake123@gmail.com";

    printf("Hello %s\n", name);
    printf("Your favorite food is %s\n", food);
```

```
    printf("Your email is %s\n", email);
    bool isOnline = true;
    bool isStudent = false;
    bool forSale = true;
    if(isOnline){
        printf("You are online");
    }
    else{
        printf("You are offline");
    }

    return 0;
}
```

本章红色单词原型：float, specifier, decimal, double, precision, string

L03 Format specifiers are easy!

yo hey everybody in this video I got to cover format specifiers in the C programming language.

哟，嘿，大家好，在这个视频中我要讲解 C 语言中的格式说明符。

Format specifiers are special tokens that begin with a percent symbol followed by a character that specifies the data type, as well as optional modifiers such as a width, precision, and flags.

格式说明符是以百分号开头，后跟一个指定数据类型的字符，以及可选修饰符（如宽度、精度和标志）的特殊标记。

Basically speaking, it's a combination of a percent sign, a character, and optional modifiers to format the output.

基本上，它是百分号、一个字符和可选修饰符的组合，用于格式化输出。

By using format specifiers, we can control how data is displayed or interpreted.

通过使用格式说明符，我们可以控制数据的显示或解释方式。

Let me give you a demonstration.

让我给你演示一下。

We've covered a little bit of format specifiers on the previous topic on variables.

我们在上一个关于变量的主题中已经稍微涉及了格式说明符。

Let's say we have an integer variable of age, set that equal to some age.

假设我们有一个整数变量 age，把它设为某个年龄。

Then we will have a float; this will be a price.

然后我们有一个 float；这将是一个价格。

Think of a price: $19.99.

想一个价格：19.99 美元。

Let's say we have a double of Pi; a double is also a floating point number, but it has double the precision; however, it takes up more memory.

假设我们有一个双精度浮点数 Pi；double 也是浮点数，但精度是 float 的两倍，不过它占用更多内存。

Let's set this equal to the first few digits of pi to keep it simple: 3.1415926535.

为了简单起见，我们把它设为 pi 的前几位数字：3.1415926535。

Then a char currency; pick unit of currency; this has to be within single quotes; I'll pick American dollars.

然后一个 char currency；选择货币单位；这必须放在单引号内；我选美元。

And an array of characters, also known as a string, maybe name; type in your name within double quotes.

以及一个字符数组，也称为字符串，比如 name；在双引号内输入你的名字。

To display these variables using printf, we'll want to use a format specifier (that percent symbol).

要使用 printf 显示这些变量，我们需要使用格式说明符（那个百分号）。

We'll display the age first: printf(" %d\n", age);

我们先显示年龄：printf(" %d\n", age);

So "d" means decimal; this will output 25.

所以 d 表示十进制；这会输出 25。

For a float, the corresponding letter for the format specifier is "f".

对于 float，格式说明符对应的字母是 "f"。

printf("%f\n", price); oh and then we need a newline character; I tend to forget, you probably will too.

printf("%f\n", price); 哦，然后我们需要换行符；我容易忘记，你可能也会。

25 and "19.990000" and a bunch of zeros afterwards.

25 和 "19.990000" 以及后面一堆零。

Alright, now for a double you have two options: we can either use

好了，对于 double 你有两个选择：我们可以

"f" or "lf" (meaning long floating point number).
用 "f" 或 "lf"（表示长浮点数）。

printf("%lf\n", pi); "f" also works.
printf("%lf\n", pi); "f" 也能用。

You may see people use one or the other.
你可能会看到人们用其中一种。

However, when we reach the topic on accepting user input, we can use a function known as scanf, where you'll want to use "lf" if we're accepting user input for a double, that is.
然而，当我们讲到接受用户输入的主题时，我们会使用 scanf 函数，如果要接受 double 类型的用户输入，你应该使用 "lf"。

So just to be consistent, for doubles I like to use "lf" for both output and input; it helps with consistency and readability.
所以为了保持一致，对于 double 我习惯在输出和输入时都使用 "lf"；这有助于一致性和可读性。

By looking at this variable, I know that this is a double, but for output purposes "f" does work too.
看到这个变量我就知道它是 double，但就输出而言 "f" 也是可行的。

You can use either; you do you.
你可以用任何一种；随你便。

Now for a char that's a single character, we'll use "%c".
对于单个字符的 char，我们使用 "%c"。

printf("%c\n", currency); I will display currency (that's the name of the variable).
printf("%c\n", currency); 我会显示 currency（那是变量的名字）。

There's our symbol.
这就是我们的符号。

And then for an array of characters (also known as a string), you'll use "%s".
然后对于字符数组（也称为字符串），你使用 "%s"。

printf("%s\n", name); that's the name of our variable.
printf("%s\n", name); 这是我们变量的名字。

And here's our name.
这就是我们的名字。

Depending on what you want to display, the percent sign is going to be followed by a given character depending on the data type of what you want to display.
根据你想显示的内容，百分号后面会跟着一个特定的字符，该字符取决于要显示的数据类型。

And here's a few of the more common data types for beginners: integers, floats, doubles, chars, and an array of characters (also known as a string).
这里有一些初学者更常见的数据类型：整数、float、double、char 和字符数组（也称为字符串）。

Now we do have a few optional modifiers.
现在我们确实有一些可选的修饰符。

So between the percent sign and the corresponding letter of the data type, there's special characters we can add to format the output.
所以，在百分号和数据类型对应的字母之间，我们可以添加特殊字符来格式化输出。

Here's an example: here's how we can set the width when we display our output.
这是一个例子：如何设置显示输出时的宽度。

This time we'll have three numbers, three integers: int num1 = 1; int num2 = 10; int num3 = 100;
这次我们有三个数字，三个整数：int num1 = 1; int num2 = 10; int num3 = 100;

We'll use a few printf statements; we're displaying integers, we need to use "%d" as the format specifier.
我们会用几个 printf 语句；我们要显示整数，需要使用 "%d" 作为格式说明符。

I'll add a newline followed by num1, then let's copy this printf statement and do the same thing for num2 and num3.
我会加上换行符再跟 num1，然后复制这个 printf 语句，对 num2 和 num3 做同样的操作。

We're displaying three integers.
我们要显示三个整数。

Now between the percent and the letter, we can type in a number; the number that we place between these two characters will set the width.
现在在百分号和字母之间，我们可以输入一个数字；放在这两个字符之间的数字会设置宽度。

For example, 3. The width specifies the minimum number of characters to print.
例如 3。宽度指定要打印的最小字符数。

So now we're printing at least three characters (1,2,3) including any spaces.
所以现在我们至少打印三个字符（1、2、3），包括任何空格。

So my second number of 10 has one space before it, and with 100, well it does take up three spaces.
所以我的第二个数字 10 前面有一个空格，而 100 本身就占了三个空格。

If I were to change this to 4, let's take a look and see what happens.
如果我把这个改成 4，我们看看会发生什么。

Now we're printing at least four spaces (1,2,3,4).
现在我们至少打印四个空格（1、2、3、4）。

You can left-justify these values by adding a negative sign before the number; now these numbers are left-justified, and we have all the space after.
你可以通过在数字前面加负号来左对齐这些值；现在这些数字左对齐了，空格都在后面。

You can also add leading zeros by preceding the number with a zero; rather than having spaces before each value, we'll have zeros.
你也可以通过在数字前面加零来添加前导零；这样每个值前面就不是空格，而是零。

Maybe num3 is negative: -100.
也许 num3 是负数：-100。

Let's reset these.
我们重置一下。

Before the letter that corresponds to the data type, if you add a plus sign, if the number is positive you'll have a plus sign; if it's negative you'll have a negative sign.
在数据类型对应的字母之前，如果你加一个加号，那么正数会显示加号，负数会显示负号。

This could be good for accounting.
这对于会计场景很有用。

Now we'll get to precision.
现在我们来讲精度。

We'll need some floating point numbers.
我们需要一些浮点数。

Let's say we have the following prices: price1 = 19.99; (float) price2 = 1.50; and price3 = 100.0;
假设我们有以下价格：price1 = 19.99; (float) price2 = 1.50; 和 price3 = 100.0;

To display some floating point numbers, we'll use the format specifier of "f" for floating point number, followed by a newline character.
要显示浮点数，我们使用格式说明符 "f" 表示浮点数，后面跟换行符。

Let's display price1, then price2, then price3.
我们显示 price1，然后 price2，然后 price3。

Here are the floating point numbers, our prices.
这些是浮点数，我们的价格。

The default behavior of C is to display six digits after the decimal when displaying a floating point number.
C 语言在显示浮点数时的默认行为是显示小数点后六位。

Before the corresponding letter for the data type, you can add a dot (or period) then the amount of digits to display after the decimal.
在数据类型对应的字母之前，你可以添加一个点（或句号），然后指定小数点后要显示的位数。

If we're working with prices, two would be good to display two digits after the decimal (for example, dollars and cents).
如果我们在处理价格，显示小数点后两位比较合适（例如美元和美分）。

So let's do ".2" for each of these.
所以我们给每个都加上 ".2"。

That looks a lot better: 19.99, 1.50, and 100.00.
这看起来好多了：19.99、1.50 和 100.00。

If this were 0.1, we would display one digit after the decimal.
如果这是 0.1，我们会显示小数点后一位。

Do keep in mind that the output will be rounded if we were to display it; so 19.99 becomes 20.0.
请记住，显示时输出会被四舍五入；所以 19.99 会变成 20.0。

Alright, so that's how to set the precision: add a dot followed by an amount of digits to display.
好了，这就是设置精度的方法：加点并跟上一个要显示的数字位数。

And with what we've learned, you can combine precision and flags together.
结合我们学到的，你可以把精度和标志组合在一起。

Along with our precision of ".2", I will also set the minimum amount of characters to display (the minimum width to display).
除了精度 ".2"，我还会设置要显示的最小字符数（最小显示宽度）。

I'll set that to be 7, and then I'll add a plus sign, so any positive numbers will have that plus sign preceding the number.
我把它设为 7，然后加一个加号，这样任何正数前面都会有加号。

So with format specifiers along with the corresponding data type, you can set the width, precision, and flags.
所以通过格式说明符和对应的数据类型，你可以设置宽度、精度和标志。

We'll have more practice with this in the future.
我们以后会有更多练习。

And well, everybody, those are format specifiers in C.
嗯，各位，这就是 C 语言中的格式说明符。

本章红色单词原型：format specifier, decimal, float, long float, character, string

```
#include <stdio.h>
```

```c
int main() {
    // Format specifier = Special tokens that begin with a % symbol,
    //                                followed by a character that specifies the data
type
    //                                and optional modifiers (width, precision, flags).
    //                                They control how data is displayed or
interpreted.
    int age = 25;
    float price = 19.99;
    double pi = 3.14159265;
    char currency = '$';
    char name[] = "Bro Code";
    printf("%d\n", age);
    printf("%f\n", price);
    printf("%lf\n", pi);
    printf("%c\n", currency);
    printf("%s\n", name);
    // width
    int num1 = 1;
    int num2 = 10;
    int num3 = -100;
    printf("%4d\n", num1);
    printf("%4d\n", num2);
    printf("%4d\n", num3);
    // precision
    float price1 = 19.99;
    float price2 = 1.50;
    float price3 = -100.00;
    printf("%.2f\n", price1);
    printf("%.2f\n", price2);
    printf("%.2f\n", price3);
     return 0;
}
```

L04 Learn C arithmetic operators in 6 minutes!

yo hey everybody in this video we're going to cover some basic arithmetic in the C programming language, because sometimes you got to math.

Let's create a few integer variables: int x = 2 (or pick a different number, really doesn't matter), int y = 3, int z = 0.

z is going to store our result; we're going to perform various arithmetic operators on x and y and store the result within z.

For basic addition, we can write something such as z = x + y.

Since x is 2 and y is 3, 2 + 3 is 5, and we're storing that within z.

Then let's print it just to test it; we'll use printf; we're displaying an integer, we need a format specifier of %d; we will display z.

And it should be the number 5, which it is. That's addition.

哟，嘿，大家好，在这个视频中我们将讲解 C 语言中的一些基本算术运算，因为有时候你得算数。

我们创建几个整数变量：int x = 2（或者选别的数字，其实无所谓），int y = 3，int z = 0。

z 将存储我们的结果；我们将对 x 和 y 执行各种算术运算符，并将结果存储在 z 中。

对于基本的加法，我们可以写 z = x + y。

因为 x 是 2，y 是 3，2 + 3 等于 5，我们把它存储在 z 中。

然后我们打印出来测试一下；使用 printf；我们要显示一个整数，需要用格式说明符 %d；我们显示 z。

结果应该是数字 5，确实是。这就是加法。

Then we have subtraction: z = x - y. What do you think the result's going to be? -1.
We have multiplication; you use an asterisk for multiplication: z = x * y; 2 * 3 is 6.
Then division: division is going to be a little strange if we're working with integers. Let me show you what happens.
z = x / y. Normally 2 / 3 should be 0.666 repeating, right?

Well, we actually get 0. We're working with integers; integers can't store decimal portions.
If z were a floating point number, we would also need to change the format specifier to an f. Let's see what happens now.
Well, we get 0.0, but it is a floating point number.
What we're doing is integer division; if we're dividing by an integer, we can't retain that decimal portion.
So in this example, let's change y to be a floating point number, and now we get 0.666666, we have that decimal portion.
You do have to pay attention to that: if you're dividing by an integer, you won't retain that decimal portion.
Then we have the modulus operator; that's going to give you the remainder of any division.
Let's change x, y, and z back to be integers. I'll set x to be 10, we'll keep y as 3.
We'll set z = x % y. The modulus operator gives you the remainder of any division.
10 divided by 3 doesn't divide evenly; there's a remainder of 1.
Let's test that; we'll need to use the format specifier of %d for decimal.
10 doesn't divide by 3 evenly; the modulus operator gives us a remainder of 1.
It's kind of like back in school how a teacher would say "everybody get into groups of three" and there's one kid remaining; that's Steve. Steve knows what he did.
If y were 2, well 10 divides by 2 evenly. It'd be like the teacher saying "everybody in this class of 10 break up into groups of two"; then there's no remaining students left over; the remainder is 0.
The modulus operator would be great if determining if a number is even or odd, because you could just say modulus 2.
If the number is even, it should divide by 2 evenly; the remainder of 10 divided by 2 gives us a remainder of 0. But if it were 11, well there's a remainder of 1.
That's the modulus operator.
Then we have the increment and decrement operator. Increment is represented by two pluses.
x is currently 10; if I were to take x++, we would increment it by one. Let's display x; so we have 11.
Or we can decrement with minus minus: x--; x is now 9.
You see increment and decrement operators a lot when working

然后是减法：z = x - y。你觉得结果会是什么？-1。
然后是乘法；乘法用星号：z = x * y；2 乘以 3 等于 6。
然后是除法：如果处理整数，除法会有点奇怪。我演示给你看。
z = x / y。通常 2 除以 3 应该是 0.666…，对吧？
实际上我们得到 0。我们用的是整数；整数不能存储小数部分。
如果 z 是浮点数，我们还需要把格式说明符改成 f。我们现在看看会发生什么。
嗯，我们得到 0.0，但它是浮点数。
我们做的是整数除法；如果除以一个整数，我们无法保留小数部分。
所以在这个例子中，我们把 y 改成浮点数，现在得到 0.666666，我们有了小数部分。
你必须注意这一点：如果你除以一个整数，你不会保留小数部分。
然后是取模运算符；它会给出任何除法的余数。
我们把 x、y 和 z 改回整数。我把 x 设为 10，y 保持为 3。
我们写 z = x % y。取模运算符给出除法的余数。
10 除以 3 不能整除；余数是 1。
我们来测试一下；需要使用格式说明符 %d 表示十进制。
10 不能被 3 整除；取模运算符给我们余数 1。
这有点像在学校里老师会说“大家三人一组”，然后还剩一个孩子；那就是史蒂夫。史蒂夫知道他自己干了什么。
如果 y 是 2，那么 10 能被 2 整除。就像老师说“这个班 10 个人分成两人一组”；这样就没有学生剩下；余数是 0。

取模运算符在判断一个数是偶数还是奇数时非常有用，因为你可以直接用模 2。
如果数字是偶数，它应该能被 2 整除；10 除以 2 的余数是 0。但如果是 11，那么余数是 1。
这就是取模运算符。
然后是自增和自减运算符。自增用两个加号表示。
x 当前是 10；如果我写 x++，我们会把它增加 1。我们显示 x；结果是 11。
或者我们可以用自减：x--；x 现在是 9。
在处理循环时你会经常看到自增和自减运算

with loops; we need some way to count the cycles, how many times has this loop cycled. So that's one place where you'll see them.
Now with these arithmetic operators, there's also something called augmented assignment operators.
These would apply if you're reassigning the same variable.
Now let's say that x is 10; if I want to add a number to x and reassign it (for example, 2), I could say x = x + 2, and that should give me 12.
But there's a shortcut: we can say x += 2, and that would do the same thing.
This could even be a different number, like 3; now x is 13.

Now if we're subtracting, we would just have to follow a similar pattern: x = x - 3, or instead we could say x -= 3; so 10 - 3 is 7.
Then we can do multiplication: x = x * 2; x is 10 * 2, that means x will be 20, or we could say x *= 2, and that will still give us 20.

Or division: x = x / 2; that's going to give us 5, or we could say x /= 2, and that also gives us 5.
These are augmented assignment operators; it's basically a shortcut if you're reassigning the same variable.
Alright everybody, so those are some basic arithmetic operators in C.

符；我们需要某种方式来计数循环次数，这个循环循环了多少次。所以你会在那里看到它们。
对于这些算术运算符，还有所谓复合赋值运算符。
这些适用于你重新给同一个变量赋值的情况。
现在假设 x 是 10；如果我想给 x 加一个数然后重新赋值（比如 2），我可以写 x = x + 2，结果应该是 12。
但有一个快捷方式：我们可以写 x += 2，效果一样。
这甚至可以是别的数字，比如 3；现在 x 是 13。
如果是减法，我们只需遵循类似的模式：x = x - 3，或者我们可以写 x -= 3；10 - 3 等于 7。
然后是乘法：x = x * 2；x 是 10 乘以 2，这意味着 x 会变成 20，或者我们可以写 x *= 2，结果仍然是 20。
或者除法：x = x / 2；结果会是 5，或者我们可以写 x /= 2，结果也是 5。
这些就是复合赋值运算符；基本上是在重新给同一个变量赋值时的一种快捷方式。
好了各位，这些就是 C 语言中的一些基本算术运算符。

```c
#include <stdio.h>
int main() {
    // arithmetic operators + - * / % ++ --

    int x = 10;
    int y = 2;
    int z = 0;
    z = x + y;
    z = x - y;
    z = x * y;
    z = x / y;
    z = x % 2;
    // x++ increment
    // x-- decrement
    // augmented assignment operators
    x+=3;
    x-=3;
    x*=3;
    x/=3;
    printf("%d", x);
    return 0;
}
```

L05 Learn how to enter user input in C!

all right everybody in this video I'm going to show you how we

好的，各位，在这个视频中，我将向大家展示

can accept user input in C.
Now if you're using VS Code, there's a little bit of setup we have to do beforehand.
If you're using a different IDE, you can look in the timestamps and skip ahead.
If you've been following along and you're using VS Code, when we accept user input we can't do so in the output tab; we have to do so in terminal.
Here's how we can do that: go to file > preferences > settings, look up the Code Runner extension, scroll down, then check this check box "Run in Terminal".
Just for some cleanup and quality of life improvements, let's clear any previous output.
And let's do a test: I'm just going to print the word "hello" and that should run in my terminal and display the output.
Now the font size may be kind of small, and before displaying any output you may have a message such as the following; well we can actually hide that.
If you would like to do that, here's how: let's go to file > preferences > settings again, we're going to look up Code Runner (that's the extension we're using), we have to edit the JSON file.
So select "Edit in settings.json".
You could increase the font size if you would like; I think normally it's kind of small.
You can save all files before running; if this is false, I would change this to be true.
Within C, we can clear the terminal before displaying output by adding "clear &&" before GCC and everything else that comes after.
Alright, let's save everything, close it, then do a test run.

There we go, that looks pretty good.
Alright, now we're ready to begin.
We're going to declare some variables, but we'll need some variables to work with.
We'll have an int of age; now rather than assigning this a number right away (for example 25), just temporarily we're going to declare this variable but not yet assign it.
We'll have a floating point number of GPA (what's your grade point average).
A char of grade, such as a letter grade A, B, C, D, or F.

And then a name; our name will be an array of characters, also known as a string.
With our string array of name, since we're not going to be giving it a value (such as "type in your first name and last name"), we have to specify a size if we're not assigning it.
So let's say 30, meaning 30 bytes or think of it as 30 characters;

如何在 C 语言中接受用户输入。
如果你使用的是 VS Code，我们需要提前做一些设置。
如果你使用不同的 IDE，可以查看时间戳并跳过这部分。
如果你一直在跟着做并且使用的是 VS Code，接受用户输入时我们不能在输出选项卡中进行，必须在终端中进行。
方法如下：前往 文件 > 首选项 > 设置，搜索 Code Runner 扩展，向下滚动，然后勾选“Run in Terminal”。
为了清理和一些便利性改进，我们清空之前的输出。
我们来测试一下：我只打印单词 hello，它应该在终端中运行并显示输出。
现在字体大小可能有点小，而且在显示任何输出之前你可能会看到类似下面的消息；实际上我们可以隐藏它。
如果你想这样做，方法如下：再次前往 文件 > 首选项 > 设置，我们搜索 Code Runner（那就是我们正在使用的扩展），我们必须编辑 JSON 文件。
所以选择“Edit in settings.json”。
如果你想，可以增大字体大小；我觉得通常它有点小。
你可以在运行前保存所有文件；如果这个选项是 false，我会把它改成 true。
在 C 语言中，我们可以在显示输出之前通过添加 "clear &&"（放在 GCC 以及之后的所有内容之前）来清空终端。
好了，我们保存所有内容，关闭它，然后进行一次测试运行。
好了，看起来很不错。
好的，现在我们准备开始了。
我们将声明一些变量，但我们需要一些变量来操作。
我们有一个 int 类型的 age；现在不是马上给它赋一个值（比如 25），而是暂时只声明这个变量但不赋值。
我们有一个浮点数 GPA（你的平均绩点是多少）。
一个 char 类型的 grade，比如字母等级 A、B、C、D 或 F。
然后是一个 name；我们的 name 将是一个字符数组，也称为字符串。
对于我们的字符串数组 name，由于我们不打算给它赋值（比如“输入你的名和姓”），如果不赋值就必须指定大小。
我们设为 30，意味着 30 字节，或者理解为

our name has a maximum size of 30 characters, and I feel like that should be enough.
Feel free to set this to a different number if you would like.
Since we're not assigning these variables values right away, whichever blocks of memory that these variables are using, it's possible that there's some values already there from when your computer used them previously.
We can use some variables even though they're not assigned a value; other programming languages such as Java would actually prevent this behavior, while C doesn't have those guard rails.
If you were to use a variable and you don't assign a value, it can lead to undefined behavior; there may be some values left in there already from a different program on your computer.
Just for demonstration purposes, I'm going to output these variables.
So I'm going to print our age variable (it's an integer, we need %d). I'll display our age, then let's do this with GPA (that's a floating point number), our grade (which is a char, that's %c), and then our name (our name is a string, %s).

So don't do this, but just for demonstration purposes I'm going to display some variables that we haven't assigned yet; we've only declared them and allocated some space in memory to store some values.
So here's what we got: we have undefined behavior. Oh, and then I need to add some newline characters because I forgot.

Alright, let's see what happens exactly. Well, we get undefined behavior.
My age variable is 398; GPA is 0.0. If I were to run this again, we'll get more undefined behavior. Now age is 688 and my GPA is like 3 billion.
Imagine your GPA being 3 billion on a scale of 0 to 4.

It's perfectly fine to declare some variables but not assign them a value; just be sure to not use them before assigning them a value, which is what I did here intentionally.
So what you could do (although it's not strictly necessary) when we declare these variables, we can assign them some values in case we accidentally use them before assigning them; it won't lead to undefined behavior.
Just as a best practice, let's assign these to zero or some equivalent value.
So with our GPA, since it's a floating number, 0 is fine, but just for clarity I'm going to set this to be 0.0f; this tells you and other programmers that our GPA should be a floating point number and not simply an integer.
With our grade, what sort of character should we pick? If we're

30 个字符；我们的名字最大长度为 30 个字符，我觉得应该够了。
如果你愿意，可以随意设置成别的数字。
由于我们没有立即给这些变量赋值，这些变量所使用的内存块中可能已经存在之前计算机使用过的一些值。

即使没有赋值，我们也可以使用某些变量；其他编程语言如 Java 实际上会阻止这种行为，而 C 语言没有这些防护措施。

如果你使用一个变量但没有给它赋值，可能导致未定义行为；你的计算机上可能已经有其他程序留下的值。
仅仅为了演示目的，我将输出这些变量。

所以我会打印 age 变量（它是整数，需要 %d），显示 age，然后对 GPA 做同样操作（它是浮点数），然后是 grade（它是 char，用 %c），再然后是 name（name 是字符串，%s）。
所以不要这样做，但仅仅为了演示，我将显示一些我们还没有赋值的变量；我们只声明了它们，并在内存中分配了一些空间来存储值。

这就是我们得到的结果：我们有未定义行为。
哦，然后我需要添加一些换行符，因为我忘了。
好了，我们看看究竟会发生什么。嗯，我们得到了未定义行为。
我的 age 变量是 398；GPA 是 0.0。如果我再次运行，我们会得到更多未定义行为。现在 age 是 688，我的 GPA 大约是 30 亿。
想象一下你的 GPA 在 0 到 4 的范围内是 30 亿。
声明一些变量但不赋值是完全没问题的；只是要确保在赋值之前不要使用它们，这正是我在这里故意做的。
所以你可以做的是（虽然并不是严格必需的）：当我们声明这些变量时，我们可以给它们赋一些值，这样万一我们在赋值之前意外使用了它们，也不会导致未定义行为。
作为一种最佳实践，我们把这些变量赋值为 0 或某个等效值。
对于 GPA，由于它是浮点数，0 没问题，但为了清晰起见，我把它设为 0.0f；这告诉你和其他程序员，我们的 GPA 应该是浮点数，而不是简单的整数。
对于 grade，我们应该选什么样的字符？如果

simply just trying to clear out our variable and it's a character, we can use a null terminator within single quotes; we will use '\0'.
This is a null terminator character; whatever values already within our variable of grade, we'll set it to be a null terminator to effectively clear it out.
With our array of characters of name, we can set this to be an empty string, but it'll still take up 30 characters (30 bytes).

Now let's print these variables, and we no longer have that undefined behavior. Let me just do a couple test runs; yeah, it all looks good.
Although not strictly necessary, you may want to consider assigning your variables right away, because if you access them before assigning them it can lead to undefined behavior, just because there may be some leftover values from a previous program.
Alright, we have our variables; now how do we put stuff within them?
Here's how: rather than using printf, we're going to use a function called scanf to get user input.
Let's get our age first: we'll need %d for decimal, comma, space, and then our variable age. And "&" is the address-of operator; basically we're saying "at the address of our variable age, we're going to stick a value within here".
So let's do a test run: within our terminal there is no prompt; if a user is running this they'll be able to type in something, type in an age, and we display our age.
However, we should add a prompt to let the user know what they have to type in; if I'm looking at this it's just a blank screen, I don't know that I need to type in something.
Preceding our scanf function, we'll use a printf function. Then we will prompt the user to "Enter your age".
Let's run this again: "Enter your age", and then I can type in an age, and we get our output; my variable of age contains the number 25. We have successfully entered user input.
Let's type in our GPA this time: we'll need a prompt (printf), let me scroll down a little bit; we will prompt the user to "Enter your GPA" (your grade point average).
To accept user input we'll use scanf; we're accepting a floating point number, we'll need %f, followed by the address-of operator, our GPA variable.
At the address of your GPA variable, stick in a value using scanf.

Let's test it: we have our age, let's say I'm 25; then to your GPA, let's enter in a solid 3.1.
Okay, our age is 25, our GPA is 3.1 and a bunch of zeros, and you can limit that by setting the precision when we display it; I'll do that now: %.2f.

我们只是想清空变量，而且它是一个字符，我们可以使用单引号内的空终止符；我们使用'\0'。
这是一个空终止符字符；无论 grade 变量中原来有什么值，我们把它设为空终止符，从而有效地清空它。
对于字符数组 name，我们可以把它设置为空字符串，但它仍然会占用 30 个字符（30 字节）。
现在打印这些变量，我们不再有未定义行为。我进行几次测试运行；是的，看起来都很好。

虽然不是严格必需的，但你可能要考虑立即给变量赋值，因为如果在赋值之前访问它们，可能会导致未定义行为，只是因为可能有之前程序留下的值。

好了，我们有变量了；那么怎么把内容放进去呢？
方法如下：我们不是使用 printf，而是使用一个叫 scanf 的函数来获取用户输入。
我们先获取 age：我们需要 %d 表示十进制，逗号，空格，然后是我们的变量 age。& 是取地址运算符；基本上我们在说“在我们的变量 age 的地址处，我们要放一个值进去”。
我们来测试运行一下：在终端中没有提示；如果用户运行这个程序，他们可以输入一些东西，输入一个年龄，然后我们显示年龄。
然而，我们应该添加一个提示，让用户知道他们需要输入什么；如果我看这个屏幕，它只是一片空白，我不知道我需要输入东西。
在我们的 scanf 函数之前，我们使用 printf 函数。然后我们提示用户“Enter your age”。
再次运行："Enter your age"，然后我可以输入一个年龄，我们就得到了输出；我的 age 变量包含数字 25。我们成功输入了用户输入。
这次我们输入 GPA：我们需要一个提示（printf），我往下滚动一点；我们提示用户“Enter your GPA”（你的平均绩点）。
为了接受用户输入，我们使用 scanf；我们要接受一个浮点数，需要 %f，然后是取地址运算符和我们的 GPA 变量。
在你的 GPA 变量的地址处，用 scanf 放入一个值。
我们测试一下：输入年龄，假设我 25 岁；然后对于 GPA，我们输入一个扎实的 3.1。
好的，年龄是 25，GPA 是 3.1 加一堆零，你可以通过设置显示的精度来限制它；我现在就做：%.2f。

Enter your age, "Enter your GPA". Alright, here's our age, here's our GPA.
Now let's set our grade: again we'll need a prompt (printf "Enter your grade"), perhaps it's a grade on an exam or a test of some sort.
scanf, we're accepting a character, we need %c for character, at the address of our grade variable, insert a character.
And then we'll print it. Let's see what happens: 25, 3.2. Okay, now check this out: I wasn't given an opportunity to type in a letter grade; my program spits out as output my age variable and my GPA.
Here's the input buffer problem that we're encountering.
After entering our GPA variable, there's a newline character within the input buffer. When we read that floating point number, that newline character is still within the input buffer, and then we're typing in a character for our letter grade.
Our C program picked up that newline character and assigned it to the grade variable. We need to clear that input buffer before assigning our grade variable.
A shortcut to clear that newline character is with scanf: when we read that character for our grade, add a space before the percent sign.
This tells our C program to skip over that newline character. So that's a little shortcut that you can do. Let's test it out.
Enter your age, "Enter your GPA", "Enter your grade". I'll say B.
And here's our output: 25, 3.2, and the character B.

Now we have to assign our name. We could use scanf, but there is a better option. But let's try and use scanf first to see what happens exactly.
We'll type in "Enter your first name" (not your full name, we don't want any spaces).
scanf, we are accepting a string, we need a format specifier of %s, at the address of our name, then we will print our name.

Okay, let's see what happens: enter your age, enter your GPA, enter your grade, enter your first name. I'll just type in the first name of my YouTube channel.
Alright: 25, 3.2, B, and then I get my first name.
Let's change this around: instead of "Enter your first name", say "Enter your full name including a space". Check this out.
Enter your age, enter your GPA, enter your grade, enter your full name. Type in your first name and your last name.
So we only get our first name. That's because scanf can't read any whitespaces.
This is because scanf will stop reading after encountering any spaces. If you were to type in your full name, we only end up storing your first name.
A solution to this is to use a different function named fgets (file

Enter your age, "Enter your GPA"。好了，这是年龄，这是 GPA。
现在我们来设置 grade：再次需要一个提示（printf "Enter your grade"），也许是某次考试或测验的成绩。
scanf，我们要接受一个字符，需要 %c，在 grade 变量的地址处插入一个字符。
然后我们打印它。我们看看会发生什么：25，3.2。好的，现在注意：我没有机会输入字母等级；程序直接输出了我的 age 变量和 GPA。
这就是我们遇到的输入缓冲区问题。
在输入 GPA 变量之后，输入缓冲区中有一个换行符。当我们读取那个浮点数时，那个换行符仍然留在输入缓冲区中，然后我们为字母等级输入一个字符。
我们的 C 程序拾取了那个换行符并将其赋值给了 grade 变量。我们需要在给 grade 变量赋值之前清空输入缓冲区。
清除那个换行符的一个快捷方式是在 scanf 中：当我们为 grade 读取字符时，在百分号之前加一个空格。
这告诉我们的 C 程序跳过那个换行符。这是你可以做的一个小技巧。我们来测试一下。
Enter your age, "Enter your GPA", "Enter your grade"。我输入 B。这是我们的输出：25，3.2，以及字符 B。
现在我们来给 name 赋值。我们可以使用 scanf，但有更好的选择。但我们先试试用 scanf，看看究竟会发生什么。
我们输入 "Enter your first name"（不是全名，我们不想要任何空格）。
scanf，我们要接受一个字符串，需要格式说明符 %s，放在 name 的地址处，然后打印 name。
好的，看看会发生什么：输入年龄，输入 GPA，输入等级，输入名字。我只输入我 YouTube 频道名的名字部分。
好的：25, 3.2, B，然后我得到了我的名字。
我们改一下：把“输入你的名”改成“输入你的全名（包含空格）”。看看这个。
输入年龄，输入 GPA，输入等级，输入全名。输入你的名和姓。
所以我们只得到了名字。那是因为 scanf 不能读取任何空白字符。
这是因为 scanf 在遇到任何空格后会停止读取。如果你输入全名，我们最终只存储了你的名。
解决方案是使用另一个名为 fgets 的函数（文

get string).
But within the set of parentheses we have to change a few things around. We'll enter: fgets(name, sizeof(name), stdin);
We'll pass in our variable name, then the size of the input (our input for name is 30), and then type "stdin" meaning standard input.
Now if you were to change the size of name, let's say we change it to 50, you would also have to do that here manually.
A little trick that we can do is calculate it: we can call the sizeof function and then pass in our variable of name.
So if the size is 50, we can calculate it, then we don't have to type it in manually every time.
Okay, let's see what we're working with. We should be able to type in a first name and a last name including any spaces.
Enter your age, enter your GPA, enter your grade. We're still encountering that problem where there's a newline character within the buffer.
We need to clear out that newline character before typing in our name. Now with the fgets function, we don't have the luxury of adding a space before the format specifier to skip over any whitespaces.
There's a separate function for that; we can call the getchar function that can also clear the newline character within the input buffer.
Let's try this again: enter your age, enter your GPA, enter your grade, enter your full name. This is good; we have our integer, our float, our char, and our string including any whitespace for the last.
Now for the last part of this lesson, what if we were to move this print statement preceding the other print statements?
We'll type in everything again: 25, 3.2, B, type in your full name. We have our full name, then we have a newline character followed by the rest of our print statements.
Why is this happening exactly? Why do we have a newline here?

The fgets function reads a whole line of input; when you hit enter to submit that data, it includes that enter key; that's why there's a newline after we display our name.
We would need some way to get rid of that newline character within our name variable.
Here's one way to do that. This might be a little advanced, but in the future it's going to make more sense.
We're going to need the help of a header file: we will #include <string.h>.
This header file for strings provides a lot of useful functions related to working with strings.
Following the fgets function, we're going to take our name variable, add a set of square brackets, and we need the length of our name.

件获取字符串）。
但在括号内我们需要改变一些东西。我们输入：fgets(name, sizeof(name), stdin);
我们传入变量 name，然后是输入的大小（name 的输入大小是 30），然后输入 stdin，表示标准输入。
如果你改变了 name 的大小，比如改成 50，你也需要在这里手动修改。
一个小技巧是计算它：我们可以调用 sizeof 函数，然后传入 name 变量。
所以如果大小是 50，我们可以计算它，这样就不必每次都手动输入了。
好的，我们看看效果如何。我们应该能够输入包含空格的姓和名。
输入年龄，输入 GPA，输入等级。我们仍然遇到缓冲区中有换行符的问题。

在输入名字之前，我们需要清除那个换行符。对于 fgets 函数，我们不能像 scanf 那样通过在格式说明符前加空格来跳过空白字符。

有一个单独的函数可以做到；我们可以调用 getchar 函数，它也可以清除输入缓冲区中的换行符。
我们再试一次：输入年龄，输入 GPA，输入等级，输入全名。很好；我们得到了整数、浮点数、字符以及包含空格的字符串。

本课的最后一件事：如果我们将这个打印语句移到其他打印语句之前会怎样？
我们再次输入所有内容：25, 3.2, B，输入全名。我们得到了全名，然后是一个换行符，接着是其余的打印语句。
这到底是为什么？为什么这里会有一个换行符？
fgets 函数读取一整行输入；当你按回车提交数据时，它包含了那个回车键；这就是为什么在显示名字之后有一个换行符。
我们需要某种方法去掉 name 变量中的那个换行符。
这里有一种方法。这可能有点高级，但在将来会更有意义。
我们需要一个头文件的帮助：我们添加 #include <string.h>。
这个字符串头文件提供了许多与字符串操作相关的有用函数。
在 fgets 函数之后，我们获取 name 变量，加一对方括号，然后需要 name 的长度。

Using that string header file, there's a built-in function of strlen (string length); this is a function, we'll pass in our name. This will return the length of our name.	使用那个字符串头文件，有一个内置函数 strlen（字符串长度）；这是一个函数，我们传入 name。它会返回 name 的长度。
Minus one, we set the last character equal to a null terminator, and then that should work.	减一，我们将最后一个字符设置为空终止符，这样应该就可以了。
Let's test this program one last time: 25, 3.2, B, type in your full name. And that newline character is removed. We have our full name, our age, our GPA, and our letter grade.	我们最后一次测试这个程序：25, 3.2, B，输入全名。那个换行符被移除了。我们得到了全名、年龄、GPA 和字母等级。
Alright everybody, we have covered a lot of stuff today. If this is a little overwhelming, don't worry; we have a lot of practice projects coming up where we will review a lot of this. We'll take it step by step.	好了各位，今天我们涵盖了很多内容。如果有点难以消化，别担心；我们即将有很多练习项目，我们会在其中复习很多内容。我们会一步一步来。
And well, everybody, that's how to accept user input in C.	嗯，各位，这就是如何在 C 语言中接受用户输入。

```c
#include <stdio.h>
#include <string.h>
int main() {
    int age = 0;
    float gpa = 0.0f;
    char grade = '\0';
    char name[50] = "";
    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Enter your gpa: ");
    scanf("%f", &gpa);
    printf("Enter your grade: ");
    scanf(" %c", &grade); // The space before %c is to consume any leftover \n.
    getchar(); // clears \n from input buffer
    printf("Enter your full name: ");
    fgets(name, sizeof(name), stdin);
    name[strlen(name) - 1] = '\0'; //changes last char from \n to \0
    printf("%s\n", name);
    printf("%d\n", age);
    printf("%.2f\n", gpa);
    printf("%c\n", grade);
    return 0;
}
```

L06Let's code a shopping cart program with C programming

hey what's going on everybody in this video we're going to create a shopping cart program using the C programming language.	嘿，大家好，在这个视频中，我们将用 C 语言创建一个购物车程序。
A user can type in the name of an item, a price, and a quantity; this will calculate a total for you.	用户可以输入商品名称、价格和数量；程序会为你计算总价。
Let's get started.	让我们开始吧。
Okay, let's get started everybody.	好的，各位，我们开始吧。
Usually when creating a project, a good first step is to declare all the variables you'll need.	通常在创建项目时，一个好的第一步是声明你需要的所有变量。
We'll need to hold the name of an item; this can be an array of characters, otherwise known as a string.	我们需要保存商品名称；这可以是一个字符数组，也称为字符串。

We will create an array of characters named item.	我们将创建一个名为 item 的字符数组。
What should be the maximum number of characters for our item? Let's say 50; that's a good number.	我们的商品名称最多多少个字符呢？设为 50；这个数字不错。
What I like to do is initialize all my variables right away, even if it's to zero or some sort of empty value.	我喜欢做的是立即初始化所有变量，即使只是赋值为零或某种空值。
I'll set my item to be an empty string; this is just so we avoid undefined behavior in case we use one of these variables before they're assigned.	我会把 item 设为空字符串；这只是为了避免在赋值之前使用这些变量时出现未定义行为。
We'll need a price; our price can be a floating point number, float price, and I will set this to be 0.0f.	我们需要价格；价格可以是浮点数，float price，我将其设为 0.0f。
If it were a double you can omit the 'f', but this tells you and other developers that this should be a floating point number.	如果是 double 可以省略 f，但这告诉你和其他开发者它应该是一个浮点数。
Then a quantity: how many items are we buying? int quantity; I will set that to be zero.	然后是数量：我们要买多少件？int quantity；我将其设为零。
And a currency: our currency is a char (a character); pick a unit of currency, I'll pick American dollars.	然后是货币单位：currency 是一个 char（字符）；选一个货币单位，我选美元。
And then a total: that will be a floating point number, float total = 0.0f.	然后是总价：这将是一个浮点数，float total = 0.0f。
Alright, we have our variables.	好了，我们有变量了。
Now we need some prompts; we're going to ask a user what kind of item they're buying.	现在我们需要一些提示；我们将询问用户购买什么商品。
We'll need a printf statement: "What item would you like to buy?"	我们需要一个 printf 语句："What item would you like to buy?"
Then another prompt: "What is the price of each item?"	然后另一个提示："What is the price of each item?"
Between each print statement we'll accept user input; I just want to get these prompts out of the way.	在每个打印语句之间我们将接受用户输入；我只是想先把这些提示写好。
What is the price for each item? Let's just say "each".	每个商品的价格是多少？ 我们就说 "each"。
Then we need a quantity; we'll ask the user how many they're buying: "How many would you like?"	然后需要数量；我们询问用户要买多少："How many would you like?"
Alright, we have our three prompts.	好了，我们有三个提示了。
Then we need to accept user input.	然后我们需要接受用户输入。
So with our item, it's a string; we'll need to use fgets.	对于我们的 item，它是字符串；我们需要使用 fgets。
Within the fgets function, we will place our item variable; we need the size of our array of characters (50), but it would be better if we were to use the sizeof function to calculate it.	在 fgets 函数中，我们放入 item 变量；我们需要字符数组的大小（50），但最好使用 sizeof 函数来计算。
Return the size of item just in case this number changes; we don't have to manually change it.	返回 item 的大小，以防这个数字改变；我们不需要手动修改。
Then we have stdin (standard input).	然后我们使用 stdin（标准输入）。
Then we need the price; the price is a floating point number. How do we get a floating point number from the user? We can use scanf.	然后我们需要价格；价格是浮点数。如何从用户那里获取浮点数？我们可以使用 scanf。
We need a format specifier for a float, that would be %f; then use the address-of operator, our price variable.	我们需要一个 float 的格式说明符，即 %f；然后使用取地址运算符，我们的 price 变量。

Then we need the quantity; how many are we buying? Quantity is an integer; we'll use scanf to get an integer.	然后我们需要数量；我们要买多少？quantity 是整数；我们将使用 scanf 获取整数。
The format specifier is %d for decimal, at the address of quantity.	格式说明符是 %d（十进制），取 quantity 的地址。
Then we will assign it.	然后我们赋值。
Then we have a total; our total is going to be total = price * quantity.	然后我们有总价；total = price * quantity。
Let's say I'm buying three pizzas and each pizza is $10; well, 10 * 3 should be 30.	假设我买三个披萨，每个披萨 10 美元；那么 10 乘以 3 应该是 30。
Let's do a test: I'm going to use printf, we'll display our total, %f for floating point number; we will display our total variable.	我们来测试一下：我将使用 printf，显示 total，%f 表示浮点数；我们将显示 total 变量。
What item would you like to buy? I would like to buy a pizza; I'm going to say "pizza".	你想买什么商品？ 我想买披萨；我说 "pizza"。
$10.1 for testing purposes. How many would you like? Three pizzas.	测试用价格设为 10.1 美元。你想要几个？三个披萨。
Alright, currently our total is 30.300001. Well, we can set the precision of a floating point number with the format specifier.	好的，目前我们的总价是 30.300001。嗯，我们可以通过格式说明符设置浮点数的精度。
Let's add ".2" to display two digits after the decimal, so that the output is clearly in cents.	我们加上 ".2" 来显示小数点后两位，这样输出就清楚地显示为美分。
Let's try that again: I will buy a pizza; the price is $10.1; and I will buy three pizzas.	我们再试一次：我买一个披萨；价格是 10.1 美元；我买三个。
30.30.	30.30。
Now let's add our currency. So with our output, we'll add a format specifier for character (%c) followed by the total.	现在加上货币单位。所以输出时，我们添加一个字符的格式说明符 %c，然后是总价。
So we have our character of currency and our total.	这样我们就有了货币字符和总价。
Let's do another test run: I would like to buy a pizza; each pizza is $10.1; and I would like to buy three.	再来一次测试运行：我想买一个披萨；每个披萨 10.1 美元；我想买三个。
My total is $30.30, and we do have that unit of currency beforehand.	我的总价是 30.30 美元，而且我们确实在前面有货币单位。
So when printing variables, you can list multiple format specifiers all in the same line if you need to, which is what we did here; we're displaying our currency and our total, our char and our floating point number.	所以在打印变量时，如果需要，你可以把多个格式说明符放在同一行，我们这里就是这样做的；我们显示了货币和总价，即 char 和浮点数。
Let's also print another statement. I'm actually going to add a newline character.	我们再打印一条语句。我实际上要加一个换行符。
You have bought %d %s\n — our quantity variable (format specifier for integer is %d), followed by our item name (that's a string, %s).	You have bought %d %s\n —— 我们的 quantity 变量（整数的格式说明符是 %d），然后是商品名称（字符串，%s）。
We're going to display our quantity and our item (the name of our item). Then I'm going to add a newline character.	我们将显示数量和商品名称。然后加一个换行符。
Okay, let's see what we got: I would like to buy a pizza; the price will be $12.99; I would like to buy five pizzas (it's for a party).	好的，我们看看结果：我想买披萨；价格是 12.99 美元；我想买五个披萨（为了聚会）。

You have bought 5 pizza then we have a space after, and then our total of $64.95.	You have bought 5 pizza 然后后面有空格，接着总价 64.95 美元。
Our item name has a newline character within the input buffer; we need to get rid of that.	我们的商品名称在输入缓冲区中有一个换行符；我们需要去掉它。
So when we accept our user input for our string using fgets, here's how we can remove that newline character: we'll take the name of our variable (our item, it's a string), add a set of square brackets, we need to get the length of our item.	因此，当我们使用 fgets 接受字符串输入时，去掉换行符的方法如下：取变量名（item，字符串），加一对方括号，我们需要获取 item 的长度。
We'll need the help of a header file: #include <string.h>. This header file contains functions related to strings.	我们需要一个头文件的帮助：#include <string.h>。这个头文件包含与字符串相关的函数。
What is the length of our string of item? This function will return that value: strlen (string length).	我们的 item 字符串的长度是多少？这个函数会返回该值：strlen（字符串长度）。
If it's "pizza", well "pizza" has 5 characters; then if you include a newline character, that's 6, minus one to go to the end.	如果是 "pizza"，它有 5 个字符；如果包含换行符就是 6，减一指向末尾。
We're going to set this last character equal to a null terminator character, and that should get rid of that newline character.	我们将最后一个字符设置为空终止符，这样就应该能去掉换行符。
Okay, what item would you like to buy? I would like to buy a pizza (or pizzas). What is the price for each? $12.99. How many would you like? I would like eight pizzas.	好的，你想买什么商品？我想买披萨。每个价格是多少？12.99 美元。你想要多少个？我要八个披萨。
You have bought 8 pizza — the total is $103.92.	You have bought 8 pizza —— 总价是 103.92 美元。
And I'm just going to make one change: I'm going to add an 's' to the end of my item when displaying it. I'll just add "s".	我做一个修改：显示商品名称时在末尾加一个 s。我就加 "s"。
Just so grammatically it makes sense: "I would like to buy a pizza; the price is $12.99; I would like 10 of them."	这样语法上就通顺了："我想买一个披萨；价格是 12.99 美元；我想要 10 个。"
You have bought 10 pizzas; the total is $129.90.	You have bought 10 pizzas; the total is $129.90.
You could even add "the total is" then our total too, if you would like.	你甚至可以加上 "the total is" 然后再跟总价，如果你愿意的话。
Pizza $12.99; I want 10 pizzas; you have bought 10 pizzas; the total is $129.90.	披萨 12.99 美元；我想要 10 个披萨；你买了 10 个披萨；总价是 129.90 美元。
Alright everybody, so that is a shopping cart program that you can write for practice using the C programming language.	好了各位，这就是一个你可以用 C 语言编写的购物车程序，供练习使用。

```c
#include <stdio.h>
#include <string.h>
int main() {

    // SHOPPING CART PROGRAM

    char item[50] = "";
    float price = 0.0f;
    int quantity = 0;
    char currency = '$';
    float total = 0.0f;
    printf("What item would you like to buy?: ");
    fgets(item, sizeof(item), stdin);
```

```
    item[strlen(item) - 1] = '\0';
    printf("What is the price for each?: ");
    scanf("%f", &price);
    printf("How many would you like?: ");
    scanf("%d", &quantity);
    total = price * quantity;
    printf("\nYou have bought %d %s/s\n", quantity, item);
    printf("The total is: %c%.2f", currency, total);

    return 0;
}
```

L07Let's code a mad libs game with C programming!

hey everybody in this video we're going to create an interactive Mad Libs game using C.	嘿，大家好，在这个视频中，我们将使用 C 语言创建一个交互式的 Mad Libs 游戏。
This topic serves as a practice project so we get used to accepting strings as user input.	这个主题作为一个练习项目，让我们习惯接受字符串作为用户输入。
Let's get started.	让我们开始吧。
Before we begin, we'll need to include the string header file because we'll be working with strings.	在开始之前，我们需要包含字符串头文件，因为我们要处理字符串。
#include <string.h> – mostly we just need to remove the newline character at the end of each string when we accept user input.	#include <string.h> —— 主要是为了在接收用户输入时去掉每个字符串末尾的换行符。
We'll declare the different variables we'll need.	我们将声明需要的各种变量。
The only type of variable we'll be working with in this topic are arrays of characters, also known as strings.	在这个主题中，我们唯一使用的变量类型是字符数组，也称为字符串。
Let's create a character array of noun; in the English language, a noun is a person, place, or thing.	我们创建一个名词的字符数组；在英语中，名词是人、地点或事物。
You're also going to get an English lesson here today too.	你今天在这里还会顺便上一堂英语课。
Then we'll specify a size; we'll say 50 for maximum size of 50 characters; that should be more than enough.	然后我们指定大小；设为 50，最大 50 个字符；这应该绰绰有余。
I'll set these to be empty strings right away.	我会立即把这些设置为空字符串。
A noun is a person, place, or thing.	名词是人、地点或事物。
Then a verb; a verb is an action like running.	然后是动词；动词是像“跑”这样的动作。
I will set that to be an empty string.	我会把它设为空字符串。
We'll need a few adjectives; an adjective describes something such as fast, slow, loud, quiet.	我们需要几个形容词；形容词描述某事物，比如快的、慢的、大声的、安静的。
I'll just set these all to be 50.	我就把这些都设为 50。
We'll need a few adjectives; I'll say adjective one, then two, then three.	我们需要几个形容词；我说形容词一、二、三。
And I'll just copy this.	我就复制一下。
Then we have adjective two, adjective three.	然后我们有形容词二、形容词三。
These are the variables we'll need; they're all arrays of characters (strings).	这些是我们需要的变量；它们都是字符数组（字符串）。
A user is going to fill these in.	用户将填写这些内容。
We'll need some prompts.	我们需要一些提示。
We'll use printf.	我们将使用 printf。
The first prompt we'll ask for is for an adjective; an adjective describes something: "Enter an adjective".	第一个提示是询问形容词；形容词描述某事物："Enter an adjective"。

I'll add a hint here too; let's say "an adjective is a description".	我在这里也加一个提示；比如"形容词是一种描述"。
Let's create another prompt.	我们再创建另一个提示。
The next prompt will be for a noun: "A person, place, or thing; enter a noun".	下一个提示是询问名词："人、地点或事物；输入一个名词"。
And I'll add a hint that we are looking for an animal or a person.	我会加一个提示，说我们想要一个动物或一个人。
We'll need an adjective again.	我们还需要一个形容词。
I'll copy this printf statement and just paste it.	我复制这个 printf 语句然后粘贴。
Then a verb: "A verb is an action such as running or jumping; enter a verb".	然后是动词："动词是像跑或跳这样的动作；输入一个动词"。
I want the verb to end in "ing" so that it's present tense; someone or something is currently doing something.	我希望动词以 ing 结尾，这样就是现在时；某人或某物正在做某事。
Verb ending with ing.	以 ing 结尾的动词。
And then an adjective again; again an adjective is a description; we're describing something.	然后再来一个形容词；形容词是一种描述；我们在描述某事物。
Now we need to accept user input.	现在我们需要接受用户输入。
Since we're accepting strings including whitespaces, we'll use fgets, because scanf doesn't accept any whitespaces.	由于我们接受包含空格的字符串，我们将使用 fgets，因为 scanf 不接受任何空格。
A user will enter in adjective one; that's the first argument.	用户将输入 adjective one；那是第一个参数。
Then we need the size of adjective one; we could just say 50, but it's better to calculate it just in case we change it; we don't have to manually change it here too.	然后我们需要 adjective one 的大小；我们可以直接写 50，但最好计算它，以防我们改变它；这样我们这里也不需要手动修改。
Return the size of adjective one, then we need stdin for standard input.	返回 adjective one 的大小，然后我们需要 stdin 表示标准输入。
We'll have to do the same thing for noun; we'll do a lot of copying and pasting.	对 noun 我们也得做同样的事；我们会做很多复制粘贴。
fgets(noun, sizeof(noun), stdin); then adjective again (this will be for adjective two);	然后又是形容词（这次是 adjective two）；
then our verb variable: fgets(verb, sizeof(verb), stdin); and adjective three.	然后是动词变量：fgets(verb, sizeof(verb), stdin)；以及 adjective three。
Temporarily, let's display our variables using printf; we're displaying a string, we'll need a format specifier of %s for string, then a newline.	临时地，我们用 printf 显示变量；我们显示字符串，需要格式说明符 %s，然后换行。
We'll display noun, our verb, and our adjectives (adjective 1, 2, and 3).	我们会显示 noun、verb 以及 adjectives（adjective 1、2 和 3）。
We just want to be sure that everything's working.	我们只是想确保一切正常。
Let's do a test run.	我们做一次测试运行。
An adjective describes something; I'll say "fast".	形容词描述某事物；我说 "fast"。
Enter a noun (an animal or a person); I'll say "Sonic".	输入一个名词（动物或人）；我说 "Sonic"。
Enter an adjective; "cool".	输入一个形容词；"cool"。
Enter a verb ending with ing; "running". (I forgot to add a colon and a space; I'll have to fix that.)	输入一个以 ing 结尾的动词；"running"。（我忘了加冒号和空格；我得修复一下。）
Enter an adjective; "funny".	输入一个形容词；"funny"。
Here's our output: we have Sonic, running, fast, cool, funny.	这是我们的输出：我们有 Sonic、running、fast、cool、funny。

So there is a newline character within the input buffer; we'll have to remove that. But I also need to add a colon and a space after this print statement because I forgot to do that.	所以输入缓冲区中有一个换行符；我们必须去掉它。但我还需要在这个 print 语句后面加一个冒号和一个空格，因为我忘了做。
Now we need to get rid of that newline character within the input buffer.	现在我们需要去掉输入缓冲区中的那个换行符。
So to do that, we'll take each variable (in this case adjective one, we'll start here), add a set of square brackets, call the string length function (strlen).	为此，我们取每个变量（这里从 adjective one 开始），加一对方括号，调用字符串长度函数 strlen。
This will return the length of the string; pass in our variable of adjective one, minus one; set this equal to a null terminator character.	这会返回字符串的长度；传入 adjective one 变量，减一；将其设置为空终止符。
Then we just have to repeat this process for the other variables.	然后我们只需要对其他变量重复这个过程。
We'll replace adjective one with noun; do this with adjective two; then verb; then adjective three.	我们把 adjective one 替换为 noun；对 adjective two 做同样操作；然后 verb；然后 adjective three。
Alright, let's do our test run again.	好了，我们再做一次测试运行。
Enter an adjective: "fast".	输入一个形容词："fast"。
Enter a noun: "Sonic".	输入一个名词："Sonic"。
Enter an adjective: "cool".	输入一个形容词："cool"。
Enter a verb ending with ing: "running".	输入一个以 ing 结尾的动词："running"。
Enter an adjective: "funny".	输入一个形容词："funny"。
Sonic is my noun, running is my verb, adjectives 1 through 3 are fast, cool, funny.	Sonic 是我的名词，running 是我的动词，形容词 1 到 3 是 fast、cool、funny。
Now we just have to work on our story.	现在我们只需要编写故事。
We can get rid of these print statements.	我们可以去掉这些打印语句。
We're going to write a story using printf statements, then we'll be inserting these variables at key locations.	我们将使用 printf 语句编写一个故事，然后在关键位置插入这些变量。
Feel free to write a different story if you would like, but here's one that I wrote.	如果你愿意，可以写一个不同的故事，但这是我写的一个。
We're going to be visiting a zoo.	我们将参观一个动物园。
I'll start with a newline character: "Today I went to a " – we're going to insert adjective one here.	我先加一个换行符："Today I went to a " —— 我们在这里插入 adjective one。
We'll insert a string %s, then the word "zoo", newline character.	我们插入一个字符串 %s，然后单词 "zoo"，换行符。
We'll be inserting adjective one at this location before "zoo".	我们在这个位置、"zoo" 之前插入 adjective one。
We're describing our zoo; is it expensive? Is it dirty? Is it clean?	我们在描述动物园；它昂贵吗？脏吗？干净吗？
With our second sentence, we'll say: "\nIn an exhibit I saw a " – here we'll insert our noun (our person, place, or thing), followed by a newline character.	对于第二句话，我们会说："\nIn an exhibit I saw a " —— 这里插入我们的名词（人、地点或事物），然后换行符。
We'll insert our noun variable here.	我们在这里插入 noun 变量。
printf("\n%s "); we'll insert our noun variable; we'll reuse it.	printf("\n%s "); 我们插入 noun 变量；我们会重用它。
%s was – insert an adjective – " and " – insert a verb.	%s was —— 插入一个形容词 —— " and " —— 插入一个动词。
I'll add an exclamation point and a newline.	我会加一个感叹号和一个换行符。

Within this printf statement, we have three variables to insert: our noun, adjective two, and our verb.	在这个 printf 语句中，我们有三个变量要插入：名词、形容词二和动词。
One more sentence: "I was " — insert adjective three — newline.	再一句话："I was " —— 插入形容词三 —— 换行。
Let's see our story.	我们看看故事。
Enter an adjective; I'm going to say "sussy" (like it's suspicious).	输入一个形容词；我说 "sussy"（像是可疑的）。
Enter a noun; if you've been around my channel long enough, you know that I like to poke fun at Mark Zuckerberg; I'm going to type in "Mark Zuckerberg".	输入一个名词；如果你关注我的频道足够久，你知道我喜欢开玩笑调侃马克·扎克伯格；我输入 "Mark Zuckerberg"。
Enter an adjective: "sweaty".	输入一个形容词："sweaty"。
Enter a verb ending with ing: "screaming".	输入一个以 ing 结尾的动词："screaming"。
Enter an adjective: "surprised".	输入一个形容词："surprised"。
Here's my story: "Today I went to a sussy zoo. In an exhibit I saw a Mark Zuckerberg. Mark Zuckerberg was sweaty and screaming. I was surprised."	这是我的故事："今天我去了一家可疑的动物园。在一个展区里我看到一个马克·扎克伯格。马克·扎克伯格又汗流浃背又在尖叫。我很惊讶。"
Alright, and that's our Mad Libs game, everybody.	好了，这就是我们的 Mad Libs 游戏，各位。
Do me a favor and post your story in the comments section down below, cuz I would like to read them, and I'm sure other people would too.	帮我个忙，把你的故事贴在下方的评论区，因为我想读读它们，而且我相信其他人也想读。
And well, everybody, that is a Mad Libs game written in C.	嗯，各位，这就是一个用 C 语言编写的 Mad Libs 游戏。

```c
#include <stdio.h>
#include <string.h>
int main() {

    // MAD LIBS GAME

    char noun[50] = "";
    char verb[50] = "";
    char adjective1[50] = "";
    char adjective2[50] = "";
    char adjective3[50] = "";
    printf("Enter an adjective (description): ");
    fgets(adjective1, sizeof(adjective1), stdin);
    adjective1[strlen(adjective1) - 1] = '\0';
    printf("Enter a noun (animal or person): ");
    fgets(noun, sizeof(noun), stdin);
    noun[strlen(noun) - 1] = '\0';
    printf("Enter an adjective (description): ");
    fgets(adjective2, sizeof(adjective2), stdin);
    adjective2[strlen(adjective2) - 1] = '\0';
    printf("Enter a verb (ending w/ -ing): ");
    fgets(verb, sizeof(verb), stdin);
    verb[strlen(verb) - 1] = '\0';
    printf("Enter an adjective (description): ");
    fgets(adjective3, sizeof(adjective3), stdin);
    adjective3[strlen(adjective3) - 1] = '\0';
    printf("\nToday I went to a %s zoo.\n", adjective1);
```

```
    printf("In an exhibit, I saw a %s.\n", noun);
    printf("%s was %s and %s!\n", noun, adjective2, verb);
    printf("I was %s!\n", adjective3);

    return 0;
}
```